# COMBINATOR CALCULUS: DECIDABLE TYPE CHECKING IN EXTENSIONAL TYPE THEORY

LEXI Z. AIELLO [a] AND ERIC KLAVINS [b]

[a] University of Washington, Department of Philosophy

[b] University of Washington, Electrical & Computer Engineering Department

Abstract. In this paper, we present a new approach for achieving decidable type-checking in extensional type theory. Our approach is based on affine categorical combinators. We show that type-checking is decidable in affine combinatory type theory with dependent types. We present our construction of this kind of combinatory type theory: the Dependent Combinator Calculus (DCC). We show that equality reflection holds as a computable metatheorem in DCC, while type-checking is decidable. We believe DCC is the most complete development of combinatory dependent type theory thus far: it has no contexts or variables; it is extensional by equality reflection; it is strongly-normalizing; it is consistent; and it is similar in expressivity to the Calculus of Constructions on the lambda-cube. We also present a procedure for compiling the subset of Lean 4 that maps to the base Calculus of Constructions to the calculus. DCC is formalized fully in the Lean 4 proof assistant.

## Introduction

**Results.**

- In 1, we present the dependent combinator calculus (**DCC**): our extension of the categorical combinators [CCM85] [Cur86] to extensional dependent type theory and affine logic. We have no variables and no contexts. Types are terms in DCC, as in the Calculus of Constructions with respect to the $\lambda$-cube [Bar91]. We describe our $\beta$-reduction rules and inference rules.
- In 3.1, we prove some basic properties of conversion in DCC: strong normalization, confluence, uniqueness of normal forms, and decidability of $\beta$-equality.
- In 4, we present the inference rules for DCC.
- In 4.1, we prove that type checking is decidable in DCC.
- In 4.2, we prove that subject-reduction and progress hold in DCC, and that DCC is consistent.
- In 5, we prove that extensional equality reflection holds in DCC. The proof is a computable definition, rather than a theorem in **Prop**.
- In 6, we present a procedure for compiling the subset of Lean 4 that maps to the base Calculus of Constructions to DCC.

*Key words and phrases:* combinatory logic, type theory, dependent types, Lean 4.

Preprint submitted to
Logical Methods in Computer Science

© L. Aiello and E. Klavins
Creative Commons

All of our results have been formalized in the Lean 4 proof assistant. To this date, DCC is the only calculus with extensional equality reflection and decidable type-checking. However, due to our choice of affine logic and categorical combinators, we inherit these features by construction.

## Background.

**The SK Combinator Calculus.** Previous attempts by others at combinatory dependent type theory have used the **SK** combinator calculus [Alt+23b, p. 10:14]. The SK combinators are a Turing-complete term-writing system equivalent to the $\lambda$-calculus [Alt+23a] with the rewrite rules $Kxy = x$ and $Sxyz = xz(yz)$. Good progress has been made, but past approaches were limited by the minimalism of SK. Types become very large and difficult to work with, relying on a hierarchy of helper ("meta" combinators) to encode basic types [10:17 Alt+23b, para. 2], and type parameters in the primitive combinators $S$ and $K$ were also quantified externally, rather than internally [10:17 Alt+23b, para. 2].

By contrast, DCC is based on an extension of categorical combinators (**CCL**) to dependent types and affine logic. In DCC, type arguments are internalized, rather than externally quantified and all universe levels are inferred. No hierarchy of helper ("meta" combinators) are required to support dependent types (e.g., $\Pi[Yx][Zx]$ in [10:17 Alt+23b, lines 5-7]). The only constructions required are $\Pi$-types, $\Sigma$-types, identity types, sorts, and the core computational combinators as in CCL.

**Categorical Combinators (CCL).** Categorical combinators [CCM85; Cur86] are a combinatory logic based on cartesian-closed categories (**CCC**) [nLa]. They are equivalent to the $\lambda$-calculus [p. 205 Cur86, theorem 2.20]. CCL has the following rewrite rules:

$$\textsc{Fst}\ \frac{}{Fst(x,y) \to_{\beta\text{CCL}} x} \qquad \textsc{Snd}\ \frac{}{Snd(x,y) \to_{\beta\text{CCL}} y} \qquad \textsc{Assoc}\ \frac{}{(x \circ y)z \to_{\beta\text{CCL}} x(yz)} \qquad \textsc{Id}\ \frac{}{Idx \to_{\beta\text{CCL}} x}$$

$$\textsc{Pair}\ \frac{}{\langle x,y\rangle x \to_{\beta\text{CCL}} (Mx, Nx)} \qquad \textsc{App}\ \frac{}{App(x,y) \to_{\beta\text{CCL}} xy} \qquad \textsc{Curry}\ \frac{}{\Lambda(x)yz \to_{\beta\text{CCL}} x(y,z)}$$

Cartesian-closed categories are known to be a categorical semantics for the $\lambda$-calculus [LS88]. The equivalence between CCL and the $\lambda$-calculus [p. 205 Cur86, theorem 2.20] is due to the shared correspondence [CCM85, p. 6] between CCL, cartesian-closed categories, and the $\lambda$-calculus with De Bruijn indices ([De 72]):

- Composition in CCC corresponds to the composition combinator in CCL.
- Identity morphisms in CCC correspond to the $Id$ combinator in CCL.
- De Bruijn indices correspond to binary projection in CCC (or "couple" projection in CCL) where $[\![0]\!] \coloneqq Snd$ and $[\![n+1]\!] \coloneqq [\![n]\!] \circ Fst$.
- Currying in CCC corresponds to the $\Lambda$ combinator in CCL [p. 176 CCM85, line 5], such that $[\![\lambda.M]\!] \coloneqq \Lambda([\![M]\!])$.
- Evaluation in CCC corresponds to the $App$ combinator in CCL, such that $[\![MN]\!] \coloneqq App \circ \langle [\![M]\!], [\![N]\!] \rangle$.

The correspondence between CCL and the De Bruijn-indexed $\lambda$-calculus makes them a more intuitive compilation target compared to SK, producing more legible terms. To illustrate, we contrast the compilation of church numerals zero and one from $\lambda$-calculus with De Bruijn indices to CCL and SK:

$$\begin{aligned} 0_\lambda &\coloneqq \lambda.\lambda.0 & [\![0_\lambda]\!] &= \Lambda(Snd) \\ 1_\lambda &\coloneqq \lambda.\lambda.10 & [\![1_\lambda]\!] &= \Lambda(App \circ \langle Fst, Snd\rangle) \end{aligned}$$

$$\begin{aligned} [\![0_\lambda]\!]_{\mathrm{SK}} &= K(SKK) \\ [\![1_\lambda]\!]_{\mathrm{SK}} &= S(S(KS)(S(KK)(SKK)))(K(SKK)) \end{aligned}$$

The reductions of $1_\lambda xy$, $[\![1_\lambda]\!]xy$, and $[\![1_\lambda]\!]_{\mathrm{SK}}xy$ are as such:

$$\begin{aligned} 1_\lambda xy &= (\lambda.\lambda.10)xy & [\![1_\lambda]\!]xy &= \Lambda(App \circ \langle Fst, Snd\rangle)xy \\ &= (\lambda.x0)y & &= App \circ \langle Fst, Snd\rangle(x, y) \\ &= xy & &= App(x, y) \\ & & &= xy \end{aligned}$$

$$\begin{aligned} [\![1_\lambda]\!]_{\mathrm{SK}}xy &= S(S(KS)(S(KK)(SKK)))(K(SKK))xy \\ &= S(KS)(S(KK)(SKK))x(K(SKK)x)y \\ &= KSx(S(KK)(SKK)x)(K(SKK)x)y \\ &= S(KK)(SKK)xy(K(SKK)xy) \\ &= K(SKKx)y(SKKy) \\ &= xy \end{aligned}$$

Despite their notable advantages in term legibility, more closely resembling De Bruijn-indexed $\lambda$-calculus, adapting categorical combinators to dependent types has not been attempted thus far.

0.0.1. *The Simply-Typed Categorical Combinators.* A simply-typed variant ($\mathbf{CCL_K}$) of the categorical combinators is presented in [Cur86, p. 220]. Type arguments are given as points. Type arguments can be inferred altogether in Curry, Pair, Comp, and in couples . The inference rules are as such:

$$\textsc{Fst}\ \frac{}{Fst(A,B) : A \times B \to A} \qquad \textsc{Snd}\ \frac{}{Snd(A,B) : A \times B \to B} \qquad \textsc{Id}\ \frac{}{Id(A) : A \to A}$$

$$\textsc{App}\ \frac{}{App(A,B) : ((A \to B) \times A) \to B} \qquad \textsc{Curry}\ \frac{x : A \times B \to C}{\Lambda(x) : A \to B \to C} \qquad \textsc{Pair}\ \frac{x : A \to B, y : A \to C}{\langle x, y\rangle : A \to B \times C}$$

$$\textsc{Comp}\ \frac{x : B \to C, y : A \to B}{x \circ y : A \to C} \qquad \textsc{Coup}\ \frac{x : A, y : B}{(x, y) : A \times B}$$

0.1. **Affine Logic.** Affine logic refers to any logic where duplication ("contraction") is restricted or prohibited, although such logics are usually related to linear logic [p. 173 Kop01, sec. 1, par. 1]. Linear logic is a constructive logic [Gir87] wherein terms in context must be used exactly once, except in the "exponential" modalities (! and ?) [Gir87, p. 3]:

$$\otimes\ \frac{\vdash A, C \qquad \vdash B, D}{\vdash A \otimes B, C, D} \qquad \parr\ \frac{\vdash A, B, C}{\vdash A \parr B, C} \qquad 1\oplus\ \frac{\vdash A, C}{\vdash A \oplus B, C} \qquad 2\oplus\ \frac{\vdash B, C}{\vdash A \oplus B, C} \qquad \&\ \frac{\vdash A, C \qquad \vdash B, C}{\vdash A \& B, C}$$

$$W?\ \frac{\vdash B}{\vdash ?A; B} \qquad C?\ \frac{\vdash ?A, ?A, B}{\vdash ?A, B} \qquad D?\ \frac{\vdash A, B}{\vdash ?A, B} \qquad !,\ \frac{\vdash A, ?B}{\vdash !A, ?B}$$

Weakening is retained in affine logic. Most importantly, affine logic forms a decidable fragment of first-order logic [KW84].

## 1. The Dependent Combinator Calculus

The Dependent Combinator Calculus (DCC) is a dependent type theory where types are terms which is combinatory (it has no contexts and no variables). It is based on an extension of the simply-typed categorical combinators to dependent types. DCC uses affine logic, meaning term duplication is restricted in such a way that all programs are provably terminating by syntactic size, even if they are not well-typed. We use this fact to show that DCC is strongly-normalizing, has decidable type-checking, is consistent, and that equality reflection holds as a computable metatheoretic definition.

## 2. Term Syntax

The base objects are a modified super-set of the base terms in $\mathrm{CCL_K}$. Products are replaced by $\Sigma$, arrows by $\Pi$, an equality type along its type formers is added, sorts are added, and bottom, top, and boolean types are added with the corresponding introducing terms. The syntax is given in S-expressions for clarity. A term $(E)$ is either a constant of term arity zero $(E_c)$, a parenthesized combinator $(C)$ juxtaposed with its points, or a parenthesized application of two terms.

**Definition 2.1.** Constants $(E_c)$ are sort levels (written as Sort $u$), the bottom $(\mathbb{0})$, top $(\mathbb{1})$ type and its inhabitant $(*)$, the boolean type $(\mathbb{2})$ along with its inhabitants $\mathbb{1}_{\mathbb{2}}$, and $\mathbb{0}_{\mathbb{2}}$, and an opaque "hole" term used internally in type-checking:

$$E_c \coloneqq \{\text{Sort } u, \mathbb{0}, \mathbb{1}, \mathbb{2}, *, \mathbb{1}_{\mathbb{2}}, \mathbb{0}_{\mathbb{2}}, \text{hole}(n)\}$$

where $u, n \in \mathbb{N}$.

**Definition 2.2.** Combinators are identified by the set of names $C$:

$$C \coloneqq \{\mathbb{0}.Rec, \mathbb{2}.Rec, !\,{<>}, \$\,{<>}, *, !, \&, =, Eval, Eql, ='', refl, `, Id, Fst, Snd, Swap,$$
$$\Sigma, \Sigma', \Lambda, !\Lambda, \Pi, \circ, \langle\,\rangle, , \}$$

**Definition 2.3.** A term $(E)$ is either a constant $E_c$:

$$E_c \subset E$$

or a parenthesized combinator $C$ juxtaposed by its points which are also terms:

$$\{(c\ x_0 \dots x_n) \mid c \in C, \{x_0 \dots x_n\} \subset E\} \ \subset E$$

or a parenthesized application of a list of juxtaposed terms:

$$\{(f\ x_0 \dots x_n) \mid f \in E, \{x_0 \dots x_n\} \subset E\} \subset E$$

**Example 2.4.** If $A \in E$, then $(Id\ A) \in E$.

**Example 2.5.** If $A, x \in E$, then $((Id\ A)x) \in E$.

**Example 2.6.** If $u \in \mathbb{N}$, then Sort $u \in E$.

**Example 2.7.** If $B, x, y \in E$, then $(, B\ x\ y) \in E$.

## 3. Conversion Rules

Conversion uses call-by-value operational semantics and is a modified super-set of those in CCL. Terms are reduced in any particular order of: head reductions, points of the term if it is a combinator, or the constituent function and argument if it is an application. We have global confluence, strong normalization, and uniqueness of normal forms. We do not have a recursor for equality, since DCC is extensional 5 and propositional equality coincides with definitional equality, so it is not necessary.

When $A, B \in E$, we use $A \to_\beta B$ to denote single-step $\beta$-reduction, $A \to_\beta^* B$ to denote star-step $\beta$-reduction, and $A \to_\beta^! B$ to denote $\beta$-reduction to a normal form.

**Definition 3.1.** The identity, quotation, and composition are the same as in $\mathrm{CCL_K}$:

Id
$$\frac{}{((Id\ A)x) \to_\beta x}$$

Quot
$$\frac{}{((`B\ x)y) \to_\beta x}$$

Comp
$$\frac{}{((\circ\ x\ y)z) \to_\beta (x(yz))}$$

**Definition 3.2.** Projection over couples and currying are the same as in CCL, but with type arguments added:

Fst
$$\frac{}{((Fst\ B)(, B_2\ x\ y)) \to_\beta x}$$

Snd
$$\frac{}{((Snd\ B)(, B_2\ x\ y)) \to_\beta y}$$

Curry
$$\frac{}{((\Lambda\ A\ B\ x)yz) \to_\beta x(, B\ y\ z)}$$

**Definition 3.3.** Evaluation of couples is similar to the usual recursor for $\Sigma$:

Eval
$$\frac{}{((Eval\ A\ B\ x)(, B_2\ y\ z)) \to_\beta (xyz)}$$

**Definition 3.4.** Exchange is permitted on nondependent couples as an optimization, yielding a couple quoting the first component:

Swap
$$\frac{}{((Swap\ A\ B)(, B_2\ x\ y)) \to_\beta ((`B\ A)\ y\ x)}$$

**Definition 3.5.** Boxed terms may be unboxed, mapped into banged types, and mapped over into other boxed terms:

$$\textsc{Unbox}\ \frac{}{((*\ A)(\&\ x)) \to_\beta x} \qquad \textsc{Map}\ \frac{}{((!\ <>\ \ x)(\&\ y)) \to_\beta (!\ (xy))} \qquad \textsc{Fmap}\ \frac{}{((\$\ <>\ \ x)(\&\ y)) \to_\beta (\&\ (xy))}$$

**Definition 3.6.** Boxed terms can also be curried:

$$\textsc{Curry'}\ \frac{}{((!\Lambda\ A\ B\ x)(\&\ y)(\&\ z)) \to_\beta (x(\&\ (,B\ y\ z)))}$$

**Definition 3.7.** As opposed to its usual behavior in CCL, the pair combinator consumes boxed terms and duplicates its argument so as to be fully dependent:

$$\textsc{Pair}\ \frac{}{((\langle,\rangle B\ x\ y)(\&\ z)) \to_\beta (,(Bz)\ (xz)\ (yz(xz)))}$$

**Definition 3.8.** Equality types can also be formed by the combinators $='$ and $Eql$, which index an environment determining the arguments' types:

$$\textsc{Eq'}\ \frac{}{((='\ \ B)xyz) \to_\beta (=(Bx)\ y\ z)} \qquad \textsc{Eql}\ \frac{}{((Eql\ B)(\&\ x)(\&\ y)) \to_\beta (=(Bx)\ y\ y)}$$

**Definition 3.9.** $\Sigma$-types can be formed from boxes:

$$\textsc{Sigma'}\ \frac{}{((\Sigma'\ B\ C)(\&\ x)) \to_\beta (\Sigma\ (Bx)\ (Cx))}$$

**Definition 3.10.** We have a recursor for $\mathbb{2}$:

$$\textsc{Bool Recl.}\ \frac{}{((\mathbb{2}.Rec\ B\ x\ y)z\mathbb{1}_2) \to_\beta (xz\mathbb{1}_2)} \qquad \textsc{Bool Recr.}\ \frac{}{((\mathbb{2}.Rec\ B\ x\ y)z\mathbb{0}_2) \to_\beta (yz\mathbb{0}_2)}$$

**Definition 3.11.** Finally, there are pointwise congruences for combinators and congruences for application:

$$\textsc{Point}\ \frac{c \in C \qquad i < n \qquad x_i \to_\beta x_i'}{(c\ x_0 \dots x_n) \to_\beta (c\ x_0' \dots x_n')} \qquad \textsc{Congr. Fun.}\ \frac{f \to_\beta f'}{(fx) \to_\beta (f'x)} \qquad \textsc{Congr. Arg.}\ \frac{x \to_\beta x'}{(fx) \to_\beta (fx')}$$

3.1. **Some Properties of Conversion.** DCC is provably strongly normalizing due to our choice of affine logic. DCC is also locally confluent, and thus globally confluent by Newmann's lemma [New42]. As such, normal forms are unique and $\beta$-equality is decidable. We use this fact to establish judgmental decidability.

3.1.1. *Strong-Normalization by Decreasing Measure.* We show strong normalization by defining a function $size : E \to \mathbb{N}$ such that $A \to_\beta B$ implies $size(B) < size(A)$.

**Definition 3.12.** If $e \in E_c$, then $size(e) = 1$, and if $(fx) \in E$ and $(fx)$ is an application, then $size((fx)) = size(f) + size(x)$. Otherwise, combinator sizes are defined as:

$$size((Id\ A)) = size(A) + 1 \quad size((\circ\ x\ y)) = size(x) + size(y) + 1$$
$$size((`B\ x)) = size(B) + size(x) + 1$$

$$size((,B\ x\ y)) = size(B) + size(x) + size(y) + 1$$
$$size((Fst\ B)) = size(B) + 1$$
$$size((Snd\ B)) = size(B) + 1$$
$$size((Swap\ B\ C)) = size(B) + size(C) + 2$$

$$size((=A\ x\ y)) = size(A) + size(x) + size(y) + 1$$
$$size((Eql\ B)) = size(B) + 2$$
$$size((='\ B)) = size(B) + 2$$
$$size((refl\ B)) = size(B) + 1$$

$$size((\Pi\ B\ C)) = size(B) + size(C) + 1$$
$$size((\Sigma\ A\ B)) = size(A) + size(B) + 1$$
$$size((\Sigma'\ B\ C)) = size(B) + size(C) + 1$$

$$size((!\ x)) = size(x) + 1 \qquad size((*\ A)) = size(A) + 1$$
$$size((!<>\ x)) = size(x) + 2 \qquad size(($<>\ x)) = size(x) * 5 + 1$$
$$size((\&\ x)) = size(x) * 5 + 1$$

$$size((\Lambda\ A\ B\ x)) = size(A) + size(B) + size(x) + 2$$
$$size((\langle,\rangle B\ x\ y)) = 3 * (size(B) + (2 + size(x)) + size(y) + 3)$$
$$size((!\Lambda\ A\ B\ x)) = 2 * size(A) + 6 * (size(B) + 1) + 2 * size(x) + 5$$
$$size((Eval\ A\ B\ x)) = size(A) + size(B) + size(x) + 1$$

$$size((2.Rec\ x\ y\ z)) = size(x) + size(y) + size(z) + 1$$
$$size((\mathbb{0}.Rec\ x)) = size(x) + 1$$

**Lemma 3.13.** *If $c \in C$ and $\{x_0 \dots x_n\} \subset E$, then $size(x_i) < size((c\ x_0 \dots x_n))$ for all $i < n$.*

*Proof.* By induction on the structure of expressions. □

**Theorem 3.14.** *Strong-normalization: If $A, B \in E$ and $A \to_\beta B$, then $size(B) < size(A)$.*

*Proof.* We proceed by induction on the reduction relation.

**Id :** $size(((Id\ A)x)) = size(A) + 1 + size(x) > size(x)$
**Quot:** $size(((`B\ x)y)) = size(B) + size(x) + 1 + size(y) > size(x)$
**Comp:** $size(((\circ\ x\ y)z)) = size(x) + size(y) + 1 + size(z) > size(x) + size(y) + size(z)$

**Fst :** $size(((Fst\ B)(, B_2\ x\ y))) = size(B) + 1 + size(B_2) + size(x) + size(y) + 1 > size(x)$
**Snd:** $size(((Snd\ B)(, B_2\ x\ y))) = size(B) + 1 + size(B_2) + size(x) + size(y) + 1 > size(y)$
**Curry:**

$$\begin{aligned} size(((\Lambda\ A\ B\ x)yz)) &= size(A) + size(B) + size(x) + 2 + size(y) + size(z) \\ &> size(x) + size(B) + size(y) + size(z) \end{aligned}$$

**Eval:**

$$\begin{aligned} size(((Eval\ A\ B\ x)(, B_2\ y\ z))) &= size(A) + size(B) + size(x) + 1 + size(B_2) \\ &\quad + size(y) + size(z) + 1 \\ &> size(x) + size(y) + size(z) \end{aligned}$$

**Swap:**

$$\begin{aligned} size(((Swap\ A\ B)(, B_2\ x\ y))) &= size(A) + size(B) + 2 + size(B_2) + size(x) + size(y) + 1 \\ &> size(B) + size(A) + size(y) + size(x) + 1 \end{aligned}$$

**Unbox:** $size(((*\ A)(\&\ x))) = size(A) + 1 + (size(x) * 5 + 1) > size(x)$
**Map:** $size(((!\ <>\ \ x)(\&\ y))) = size(x) + 2 + (size(y) * 5 + 1) > size((xy)) + 1$
**Fmap:** $size(((\$\ <>\ \ x)(\&\ y))) = (size(x) * 5 + 1) + (size(y) * 5 + 1) > size((xy) * 5 + 1)$
**Curry':**

$$\begin{aligned} size(((!\Lambda\ A\ B\ x)(\&\ y)(\&\ z))) &= 2 * size(A) + 6 * (size(B) + 1) \\ &\quad + 2 * size(x) + 5 + size(y) * 5 + 1 + size(y) * z + 1 \\ &> size(x) + 5 * (size(B) + size(y) + size(z)) + 1 \end{aligned}$$

**Pair:**

$$\begin{aligned} size((((\langle, \rangle B\ x\ y)(\&\ z))) &= 3 * (size(B) + (size(x) + 2) + size(y) + 3) + (5 * size(z) + 1) \\ &> 5 * (size(B) + 4 * size(z) + 2 * size(x) + size(y)) + 1 \end{aligned}$$

**Eq':**

$$\begin{aligned} size(((='\ \ B)xyz)) &= size(B) + 2 + size(x) + size(y) + size(z) \\ &> (size(B) + size(x)) + size(y) + size(z) + 1 \end{aligned}$$

**Eql:**

$$\begin{aligned} size(((Eql\ B)(\&\ x)(\&\ y))) &= size(B) + 2 + (size(x) * 5 + 1)(size(y) * 5 + 1) \\ &> (size(B) + size(x)) + size(y) + size(y) + 1 \end{aligned}$$

**Sigma':**

$$\begin{aligned} size(((\Sigma'\ B\ C)(\&\ x))) &= size(B) + size(C) + 1 + size(x) * 5 + 1 \\ &> (size(B) + size(x)) + (size(C) + size(x)) + 1 \end{aligned}$$

**Bool Recl.:**

$$\begin{aligned} size(((2.Rec\ B\ x\ y)z\mathbb{1}_2)) &= size(B) + size(x) + size(y) + 1 + size(z) \\ &> size(x) + size(z) + size(\mathbb{1}_2) \end{aligned}$$

**Bool Recr.:**

$$\begin{aligned} size(((\mathbb{2}.Rec\ B\ x\ y)z\mathbb{1}_{\mathbb{2}})) &= size(B) + size(x) + size(y) + 1 + size(z) \\ &> size(y) + size(z) + size(\mathbb{0}_{\mathbb{2}}) \end{aligned}$$

**Point:** If $(c\,x_0 \ldots x_n) \in E$ and $i < n$, then $size(x_i) < size((c\,x_0 \ldots x_n))$ 3.13. And if $x_i \to_\beta x_i'$, then, by the inductive hypothesis, $size(x_i') < size(x_i)$, so $size((c\ x_0 \ldots x_n)) > size((c\ x_0' \ldots x_n'))$.

**Congr. Fun.:** If $(fx) \in E$ and $f \to_\beta f'$, then, by the inductive hypothesis, $size(f') < size(f)$, and $size((fx)) = size(f) + size(x) > (size((f'x)) = size(f') + size(x))$.

**Congr. Arg.:** If $(fx) \in E$ and $x \to_\beta x'$, then, by the inductive hypothesis, $size(x') < size(x)$, and $size((fx)) = size(f) + size(x) > (size((fx')) = size(f) + size(x'))$.

✔ [Formalized: `Expr.Red.decreasing`] □

**Lemma 3.15.** *If $A, B \in E$ and $A \to_\beta^* B$, then $size(B) \leq size(A)$.*

*Proof.* We proceed by induction on the reduction relation: Suppose $A = B$. Then, $size(A) = size(B)$, so $size(B) \leq size(A)$. If $A \neq B$, then there exists $B_0 \in E$ such that $A \to_\beta B_0$ and $B_0 \to_\beta^* B$. So, by 3.14, $size(B_0) < size(A)$, and by the inductive hypothesis, $size(B) \leq size(B_0)$, so $size(B) \leq size(A)$.

✔ [Formalized: `Expr.decreasing?`] □

**Lemma 3.16.** *Point congruences are closed under star-step reduction: If $c \in C$ and $\{x_0 \ldots x_n\} \subset E$ and $i < n$ and $x_i \to_\beta^* x_i'$, then $(c\ x_0 \ldots x_n) \to_\beta^* (c\ x_0' \ldots x_n')$.*

*Proof.* Suppose $x_i \to_\beta^* x_i'$. Then, if $x_i = x_i'$, then $(c\ x_0 \ldots x_n) \to_\beta^* (c\ x_0' \ldots x_n')$ by reflexivity. If $x_i \neq x_i'$, then there must exist $x_{0,i}'$ such that $x_i \to_\beta x_{0,i}'$ and $x_{0,i}' \to_\beta^* x_i'$. And, by 3.14, $size(x_{0,i}') < size(x_i)$. So, by induction, $(c\ x_{0,0} \ldots x_{0,n}) \to_\beta^* (c\ x_0' \ldots x_n')$. Hence, $(c\ x_0 \ldots x_n) \to_\beta^* (c\ x_0' \ldots x_n')$ by transitivity of single-step application of the point rule.

✔ [Formalized: `Expr.aff_mk_point'`] □

**Lemma 3.17.** *Function congruence is closed under star-step reduction: If $(fx) \in E$ and $f \to_\beta^* f'$, then $(fx) \to_\beta^* (f'x)$.*

*Proof.* The proof proceeds similarly as in 3.16, but with the function congruence single-step rule.

✔ [Formalized: `Expr.congrFun'`] □

**Lemma 3.18.** *Argument congruence is closed under star-step reduction: If $(fx) \in E$ and $x \to_\beta^* x'$, then $(fx) \to_\beta^* (fx')$.*

*Proof.* The proof proceeds similarly as in 3.16, but with the argument congruence single-step rule.

✔ [Formalized: `Expr.congrArg'`] □

**Theorem 3.19.** *Local confluence: If $A, B, C \in E$ and $A \to_\beta B$ and $A \to_\beta C$, then there exists $D \in E$ such that $B \to_\beta^* D$ and $C \to_\beta^* D$.*

*Proof.* The proof proceeds by induction on $A \to_\beta B$ and parallel case analysis on $A \to_\beta C$. We handle three possible cases: $A$ is a combinator with points; $A$ is of the form $(fx)$, and $(fx)$ can choose to make a head-reduction step (that is, there exists $h \vdash A \to_\beta A'$ where $h$

is not proven by the point, or function, or argument congruence rules); or $A$ is of the form $(fx)$, but $(fx)$ has no possible head-reduction step.

**$A$ is a combinator:** Suppose $A = (c\ x_0 \dots x_n)$ where $c \in C$ and $\{x_0 \dots x_n\} \subset E$. Then, $A$ is not an application, nor can it make a head reduction step, so both $A \to_\beta B$ and $A \to_\beta C$ must be proven by the point rule. Hence, let $B = (c\ x'_0 \dots x'_n)$ and let $C = (c\ x''_0 \dots x''_n)$. Then, there exist $i, j \in \mathbb{N}$ such that $i < n$ and $j < n$ and $x_i \to_\beta x'_i$ and $x_j \to_\beta x''_j$. It suffices to show that if either $i = j$ or $i \neq j$, then there exists $(c\ y_0 \dots y_n)$ such that $(c\ x'_0 \dots x'_n) \to^*_\beta (c\ y_0 \dots y_n)$ and $(c\ x''_0 \dots x''_n) \to^*_\beta (c\ y_0 \dots y_n)$ to prove the goal.

If $i = j$, then $x_i \to_\beta x'_i$ and $x_i \to_\beta x''_i$, so by the inductive hypothesis, there exists $y_i$ such that $x'_i \to^*_\beta y_i$ and $x''_i \to^*_\beta y_i$. So, by 3.16, $(c\ x'_0 \dots x'_n) \to^*_\beta (c\ y_0 \dots y_n)$ and $(c\ x''_0 \dots x''_n) \to^*_\beta (c\ y_0 \dots y_n)$.

If $i \neq j$, then $(c\ x'_0 \dots x'_n)$ and $(c\ x''_0 \dots x''_n)$ shares all points besides $x'_i$ and $x''_j$. So, their common reduct is $(c\ y_0 \dots y_n)$, where $y_i = x'_i$ and $y_j = i''_j$. And, $(c\ x'_0 \dots x'_n) \to^*_\beta (c\ y_0 \dots y_n)$ and $(c\ x''_0 \dots x''_n) \to^*_\beta (c\ y_0 \dots y_n)$ by transitivity of the point rule.

**$A = (fx)$ and $(fx)$ can make a head step:** Let $A'$ denote the head reduct of $(fx)$ and let $c \in C$ denote the spine of $f$ (that is, the leftmost combinator which would trigger a head reduction). Then, either $A' = B = C$, in which case $B \to^*_\beta C$ and $C \to^*_\beta C$ by reflexivity, or $A' \neq B$ or $A' \neq C$ (that is, either $A \to_\beta B$ or $A \to_\beta C$ are given by congruences).

If $A \to_\beta B$ or $B \to_\beta C$ are given by congruences, then there exist $e \in E$ which is a sub-expression contained in $A$ such that there exists $e' \in E$ where $e \to_\beta e'$ and $e$ is the atomic term reduced by the congruences $A \to_\beta B$ or $A \to_\beta C$. And, since $A \to_\beta B$ or $A \to_\beta C$ are congruences rather than head-reductions, $e'$ must be a sub-expression of $B$ or $C$.

Note that if both $A \to_\beta B$ and $A \to_\beta C$ are congruences, then their atomic reducts $e$ are either the same, in which case $B \to^*_\beta C$ by reflexivity and $C \to^*_\beta C$ by reflexivity, or their atomic reducts $e$ are not the same, but $B$ and $C$ are both applications with the spine $c$. If $B$ and $C$ are both applications with spine $c$, then $B$ and $C$ reach a common reduct by congruence transitivity, followed by head-reduction in both cases.

If $A \to_\beta B$ is a congruence, but $A \to_\beta C$ is a head-reduction, then $A' = C$ and $e$ must be a sub-expression of $B$. Note that the spine $c$ of an application $(fx)$ cannot be modified by congruence, unless $c$ is the spine of an application contained in $f$ which can make a head-reduction. But since $(fx)$ can make a head step, $c$ cannot be modified by congruence. And, since $A$ is of the form $(fx)$, then $A \to_\beta B$ must be given by function congruence or argument congruence. In either case, $B$ is of the form $(gy) \in E$ where $c$ is the spine of $g$ and $g = f$ while $x \to_\beta y$ or $y = x$ while $f \to_\beta g$.

**$A \to_\beta B$ is given by argument congruence:** In case $A \to_\beta B$ is given by argument congruence, then $y$ is of the correct shape, such that $(gy)$ can make a head reduction to a reduct $B'$ and $e$ is also a sub-expression of $C$. This is due to the fact that head-reduction rules only match on combinators with points, not partial applications 3. So, there exists $D \in E$ such that $B \to_\beta D$ by

head-reduction and $C \to_\beta^* D$ by repeated congruence through $e \to_\beta e'$ under transitivity.

$A \to_\beta B$ **is given by function congruence:** In case $A \to_\beta B$ is given by function congruence, we note that the spine $c$ is preserved in $g$. Also, $e$ must either be a point of $c$ or an inner term in argument position. In case $e$ is in argument position, the same logic as above applies. In case $e$ is a point of $c$, then, we note that $(gy)$ can still make a head-reduction, as per 3. So, there exists $D \in E$ such that $B \to_\beta D$ by head-reduction and $C \to_\beta^* D$ by repeated congruence under transitivity through $e \to_\beta e'$.

If $A \to_\beta C$ is a congruence, but $A \to_\beta B$ is a head-reduction, then the same reasoning as above applies in opposite order.

$A = (fx)$ **and** $(fx)$ **cannot make a head step:** In case $(fx)$ cannot make a head step, then $A \to_\beta B$ and $A \to_\beta C$ are given by congruences. In either case, their atomic reducts $e$ are different, so by transitivity of congruence there exists $D$ such that $B \to_\beta^* D$ and $C \to_\beta^* D$.

To illustrate this approach, suppose $((Id\ A)x) \in E$ and $h_1 \vdash ((Id\ A)x) \to_\beta B$ and $h_2 \vdash ((Id\ A)x) \to_\beta C$:

**Function congruence in $h_1$ on $(Id\ A)$:** Suppose $B$ is of the form $(f'x)$ where $(Id\ A) \to_\beta f'$. Then, $f'$ must be constructed by the point rule (3.11), so there exists $A'$ such that $A \to_\beta A'$. We proceed by case-splitting on $h_2$:

**Function congruence in $h_2$ on $(Id\ A)$:** Suppose $C$ is of the form $(f_2'x)$ where $(Id\ A) \to_\beta f_2'$. Then, $f_2'$ must be constructed by the point rule (3.11), so there exists $A_2'$ such that $A \to_\beta A_2'$. So, by the inductive hypothesis, there exists $A'' \in E$ such that $A' \to_\beta^* A''$ and $A_2' \to_\beta^* A''$. And, by 3.16, $(Id\ A') \to_\beta^* (Id\ A'')$ and $(Id\ A_2') \to_\beta^* (Id\ A'')$. So, by 3.17, $((Id\ A')x) \to_\beta^* ((Id\ A'')x)$ and $((Id\ A_2')x) \to_\beta^* ((Id\ A'')x)$.

**Argument congruence in $h_2$ on $x$:** Suppose $C$ is of the form $((Id\ A)x')$ where $x \to_\beta x'$. Then, by the single-step rule $Id$ 3.1, $((Id\ A')x) \to_\beta x \to_\beta x'$ and $((Id\ A)x) \to_\beta ((Id\ A)x') \to_\beta x'$, so $B \to_\beta^* x'$ and $C \to_\beta^* x'$.

**Argument congruence in $h_1$ on $x$:** Suppose $B$ is of the form $((Id\ A)x')$ where $x \to_\beta x'$. Then, the proof is as above, except $h_1$ and $h_2$ are inverted.

**Head reduction in $h_1$ on $((Id\ A)x)$:** Suppose $B = x$. Then, if $h_2$ is a function congruence such that $(Id\ A) \to_\beta f'$, $f'$ must be of the form $(Id\ A')$ where $A \to_\beta A'$, so $B \to_\beta x$ and $C \to_\beta x$. If $h_2$ is an argument congruence such that $h \vdash x \to_\beta x'$, then $B \to_\beta x'$ by $h$ and $C \to_\beta x'$ by reflexivity.

✔ [Formalized: `Expr.Red.conf`] □

**Theorem 3.20.** *Global confluence: If $A, B, C \in E$ and $A \to_\beta^* B$ and $A \to_\beta^* C$, then there exists $D \in E$ such that $B \to_\beta^* D$ and $C \to_\beta^* D$.*

*Proof.* The proof proceeds by Newmann's lemma:

$A = B$ **and** $A = C$**:** By reflexivity, $B \to_\beta^* B$ and $C \to_\beta^* B$.

$A = B$ **but** $A \neq C$**:** Since $A = B$, $B \to_\beta^* C$, and by reflexivity, $C \to_\beta^* C$.

$A \neq B$ **and** $A = C$**:** Since $A = C$, $C \to_\beta^* B$, and by reflexivity, $B \to_\beta^* B$.

$A \neq B$ **and** $A \neq C$**:** Then, there exist $B_0, C_0 \in E$ such that $A \to_\beta B_0$ and $A \to_\beta C_0$ and $B_0 \to_\beta^* B$ and $C_0 \to_\beta^* C$, so, by 3.19, there exists $D \in E$ such that $B_0 \to_\beta^* D$

and $C_0 \to^*_\beta D$. And, due to 3.14 and 3.15, $size(B) \leq size(B_0) < size(A)$ and $size(C) \leq size(C_0) < size(A)$. So, by well-founded induction given $B_0 \to^*_\beta B$, $B_0 \to^*_\beta D$, $C_0 \to^*_\beta C$, and $C_0 \to^*_\beta D$, there exist $B_2, C_2 \in E$ such that $B \to^*_\beta B_2$ and $D \to^*_\beta B_2$ and $C \to^*_\beta C_2$ and $D \to^*_\beta C_2$. And, similarly, since $B \to^*_\beta B_2$ and $B \to^*_\beta C_2$ (via $D$), there exists $D' \in E$ such that $B_2 \to^*_\beta D'$ and $C_2 \to^*_\beta D'$, so $B \to^*_\beta D'$ and $C \to^*_\beta D'$.

✔ [Formalized: `Expr.RedRel.globally_confluent`] □

3.2. **Decision Procedure for $\beta$-Equality.** Our $\beta$-reduction rules are implemented as a total, terminating $\beta$-reduction engine. Reduction proceeds by repeated single-step reduction via a function $step? : (A : E) \to \text{Option } (\Sigma'(B : E), A \to_\beta B)$ and well-founded recursion by decreasing syntactic size 3.14. The reducer has has soundness and completeness with respect to $\to^!_\beta$. The reducer is used to decide $\beta$-equality and is central to decidable type-checking.

In applications $(f\,x) \in E$, *step*? attempts a head single-step, if it is available:

```
@[dcc_force_reduce, dcc_force_aff, aesop unsafe norm (rule_sets := [DccForceReduce])]
def step_head? (e : Expr) : Option ((e' : Expr) × (PULift.{1, 0} (Red e e'))) :=
  match e with
  | (DCC? ((Σ' _B _C) (& _x))) ⇒ .some ⟨_, ⟨.sigma⟩⟩
  | (DCC? ((2.Rec _motive _x _y) _env 1₂)) ⇒
    .some ⟨_, ⟨.ind_bool_true⟩⟩
  | (DCC? ((2.Rec _motive _x _y) _env 0₂)) ⇒
    .some ⟨_, ⟨.ind_bool_false⟩⟩
  | DCC? ((!<> _f) (& _x))                                  ⇒ .some ⟨_, ⟨.map⟩⟩
  | DCC? (($<> _f) (& _x))                                  ⇒ .some ⟨_, ⟨.fmap⟩⟩
  | DCC? ((* _α) (& _x))                                  ⇒ .some ⟨_, ⟨.unbox⟩⟩
  | DCC? ((⟨,⟩_β _M _N) (& _x))                           ⇒ .some ⟨_, ⟨.pair⟩⟩
  | (DCC? ((Eval _α _β _x) (,__β _y _z)))                 ⇒ .some ⟨_, ⟨.evalc⟩⟩
  | (DCC? ((Eql _f_α) (& _env) (& _y)))                   ⇒ .some ⟨_, ⟨.eql⟩⟩
  | (DCC? ((=' _f_α) _x _y _z))                  ⇒ .some ⟨_, ⟨.eq_mk⟩⟩
  | (DCC? ((`_β _x) _y))                                  ⇒ .some ⟨_, ⟨.quot⟩⟩
  | (DCC? ((Id _α) _x))                                   ⇒ .some ⟨_, ⟨.id⟩⟩
  | (DCC? ((Fst _α) (,_γ _a _b)))                         ⇒ .some ⟨_, ⟨.fstc⟩⟩
  | (DCC? ((Swap _B _C) (,_B₂ _a _b)))                    ⇒ .some ⟨_, ⟨.swap⟩⟩
  | (DCC? ((Snd _α) (,_γ _a _b)))                         ⇒ .some ⟨_, ⟨.sndc⟩⟩
  | (DCC? ((∘ _x _y) _z))                                 ⇒ .some ⟨_, ⟨.comp⟩⟩
  | (DCC? ((!Λ _α _β _x) (& _y) (& _z)))                  ⇒ .some ⟨_, ⟨.curr'⟩⟩
  | (DCC? ((Λ _α _β _x) _y _z)) ⇒ .some ⟨_, ⟨.curr⟩⟩
  | _ ⇒ .none
```

✔ [Formalized: `Expr.step_head?`]

If no head step is available, it attempts to step $f$, then if $f$ cannot be stepped, it attempts to step $x$. If $e$ is a combinator of the form $(c\ x_0 \ldots x_n)$ such that $c \in C$ and $\{x_0 \ldots x_n\} \subset E$, the first point $x_i$ where $i < n$ which can make a step is used:

```
@[dcc_force_reduce, dcc_force_aff, aesop unsafe norm (rule_sets := [DccForceReduce])]
def step? (e : Expr) : Option ((e' : Expr) × (PULift.{1, 0} (Red e e'))) :=
  open Dsl in match step_head? e with
  | .some h ⇒ .some h
  | .none ⇒
    match e.cases_point_arity' with
```

```
    | .inl ⟨heq⟩ ⇒
      match e with
      | DCC? (f x) ⇒
        match f.step?, x.step? with
        | .some ⟨f', ⟨h₁⟩⟩, _ ⇒ .some ⟨(DCC? (f' x)), ⟨.congrFun h₁⟩⟩
        | _, .some ⟨x', ⟨h₁⟩⟩ ⇒ .some ⟨(DCC? (f x')), ⟨.congrArg h₁⟩⟩
        | _, _ ⇒ .none
      | _ ⇒ .none
    | .inr (.inl ⟨heq⟩) ⇒
      match (e.points.get ⟨0, by simp [heq]⟩).step? with
      | .some ⟨_, ⟨h₁⟩⟩ ⇒
        .some ⟨_, ⟨.point 0 (by simp [heq]) h₁⟩⟩
      | .none ⇒ .none
    | .inr (.inr (.inl ⟨heq⟩)) ⇒
      match (e.points.get ⟨0, by simp [heq]⟩).step?, (e.points.get ⟨1, by simp [heq]⟩).step? with
      | .some ⟨_, ⟨h₁⟩⟩, _ ⇒
        .some ⟨_, ⟨.point 0 (by simp [heq]) h₁⟩⟩
      | _, .some ⟨_, ⟨h₂⟩⟩ ⇒
        .some ⟨_, ⟨.point 1 (by simp [heq]) h₂⟩⟩
      | _, _ ⇒ .none
    | .inr (.inr (.inr (.inl ⟨heq⟩))) ⇒
      match (e.points.get ⟨0, by simp [heq]⟩).step?, (e.points.get ⟨1, by simp [heq]⟩).step? with
      | .some ⟨_, ⟨h₁⟩⟩, _ ⇒
        .some ⟨_, ⟨.point 0 (by simp [heq]) h₁⟩⟩
      | _, .some ⟨_, ⟨h₂⟩⟩ ⇒
        .some ⟨_, ⟨.point 1 (by simp [heq]) h₂⟩⟩
      | _, _ ⇒
        match (e.points.get ⟨2, by simp [heq]⟩).step? with
        | .some ⟨_, ⟨h₃⟩⟩ ⇒
          .some ⟨_, ⟨.point 2 (by simp [heq]) h₃⟩⟩
        | .none ⇒ .none
    | .inr (.inr (.inr (.inr ⟨heq⟩))) ⇒
      match (e.points.get ⟨0, by simp [heq]⟩).step?, (e.points.get ⟨1, by simp [heq]⟩).step? with
      | .some ⟨_, ⟨h₁⟩⟩, _ ⇒
        .some ⟨_, ⟨.point 0 (by simp [heq]) h₁⟩⟩
      | _, .some ⟨_, ⟨h₂⟩⟩ ⇒
        .some ⟨_, ⟨.point 1 (by simp [heq]) h₂⟩⟩
      | _, _ ⇒
        match (e.points.get ⟨2, by simp [heq]⟩).step?, (e.points.get ⟨3, by simp [heq]⟩).step? with
        | .some ⟨_, ⟨h₃⟩⟩, _ ⇒
          .some ⟨_, ⟨.point 2 (by simp [heq]) h₃⟩⟩
        | _, .some ⟨_, ⟨h₄⟩⟩ ⇒
          .some ⟨_, ⟨.point 3 (by simp [heq]) h₄⟩⟩
        | _, _ ⇒ .none
termination_by (e.size, sizeOf e)
decreasing_by
  any_goals (grind)
```

✔ [Formalized: `Expr.step?`]

Reduction to a normal form proceeds by repeated single-step reduction and well-founded recursion given strong-normalization 3.14 until no step is available:

```
@[dcc_force_reduce, aesop norm (rule_sets := [DccForceReduce])]
def reduce.{u} {α : Type u} {t : α → ℕ} (r : α → α → Prop) [hR : RedRel r t]
  (e : α) : ((e' : α) × (PULift.{u + 1, 0} (Relation.ReflTransGen r e e'))) :=
  match hR.run e with
```

```
  | .some ⟨e', ⟨h⟩⟩ ⇒
    have _size_ : t e' < t e := hR.decreasing h
    let ⟨e'', ⟨h₂⟩⟩ := reduce r (t := t) e'
    ⟨e'', ⟨.trans (.single h) h₂⟩⟩
  | .none ⇒ ⟨e, ⟨.refl⟩⟩
termination_by t e
```

✔ [Formalized: `Expr.reduce`]

**Theorem 3.21.** *Semi-completeness of single-step reduction: If $A, B \in E$ and $A \to_\beta B$, then there exists $B' \in E$ and $h' \vdash A \to_\beta B'$ such that step?$A$ = some $\langle B, h' \rangle$.*

*Proof.* Suppose $A \to_\beta B$. The proof proceeds by induction on $A$ and analysis of the definition of *step*?.

**$A$ can make a head step:** If $A$ can make a head step, then there exists some $B' \in E$ such that *step*? $A$ = some $\langle B', h \rangle$ where $h$ is proven by the corresponding single-step rule 3.

**$A$ is of the form $(fx)$ but $A$ cannot make a head step:** If $A$ cannot make a head step, then $A \to_\beta B$ must be given by either function congruence or argument congruence. Suppose there exists $f' \in E$ such that $f \to_\beta f'$. Then, by the inductive hypothesis, there exists $f_2' \in E$ such that $h \vdash f \to_\beta f_2'$ and *step*? $f$ = some $\langle f_2', h \rangle$. So, *step*? $(fx)$ = some $\langle (f_2' x), \text{congrFun } h \rangle$. Suppose $f$ is in normal form. Then, there exists $x' \in E$ such that $x \to_\beta x'$. So, by the inductive hypothesis, there exists $x_2' \in E$ such that $h \vdash x \to_\beta x_2'$. And, since $f$ is in normal form, then *step*? $f$ = none, since a contradiction is derived if *step*? $f$ = some $\langle f', h \rangle$. So, *step*? $(fx)$ = some $\langle (fx_2'), \text{congrArg } h \rangle$.

**$A$ is of the form $(c\ x_0 \dots x_n)$ where $c \in C$ and $\{x_0 \dots x_n\} \subset E$:** Note that *step*? $A$ = none if *step*? $x_i$ = none for all $i < n$. But, since $A$ is a combinator, $A \to_\beta B$ must be given by the point rule. So, there exists some $j < n$ such that $x_j \to_\beta x_j'$. And, by the inductive hypothesis, there exists $x_j''$ and $h \vdash x_j \to_\beta x_j''$ such that *step*? $x_j$ = some $\langle x_j'', h \rangle$. Hence, there must exist $B' \in E$ and $h \vdash A \to_\beta B'$ such that *step*? $(c\ x_0 \dots x_n)$ = some $\langle B', h \rangle$.

✔ [Formalized: `Expr.semi_complete`] □

**Theorem 3.22.** *Reducts given by reduce are normal forms: If $A, B \in E$ and $h \vdash A \to_\beta^* B$ and reduce$A = \langle B, h \rangle$, then $B$ is a normal form.*

*Proof.* The proof proceeds by well-founded recursion with respect to syntactic size and analysis of the definition of *reduce*. If *step*?$A$ = none, then $A$ is a normal form, since otherwise, *step*?$A \neq$ *none* by 3.21. If *step*?$A$ = some$\langle B_0, h_0 \rangle$ such that $h_0 \vdash A \to_\beta B_0$, then $size(B_0) < size(A)$ by 3.14. So, by well-founded recursion, $B_0$ is a normal form.

✔ [Formalized: `Expr.reduce_norm`] □

**Theorem 3.23.** *Completeness of $\beta$-reduction with respect to normal forms: If $A, B \in E$ and $h \vdash A \to_\beta^! B$, then reduce$(\to_\beta)A = \langle B, h \rangle$.*

*Proof.* This fact is shown by induction on $A$ with respect to size and analysis of the definition of *reduce*:

*step*?$A$ = **none:** If *step*? $A$ = none, then $A$ can be shown to be in normal-form: suppose for the sake of contradiction that $A \to_\beta B$. Then, by semi-completeness 3.21, there exist $B' \in E$ and $h \vdash A \to_\beta B'$ such that *step*? $A$ = some $\langle B', h \rangle$. Hence, a contradiction is

derived, so $A$ is in normal form. So, $h$ must be proven by reflexivity. That is, $A = B$. Therefore, $reduce(\rightarrow_\beta)A = \langle B, refl \rangle$.

*step*?$A = \langle B', h' \rangle$**, where** $h' \vdash A \rightarrow_\beta B'$**:** If a step can be made, then by confluence 3.20, there exists $D \in E$ such that $h_1 \vdash B' \rightarrow_\beta^* D$ and $h_2 \vdash B \rightarrow_\beta^* D$. But since $B$ is a normal form, $B = D$, so $B' \rightarrow_\beta^* B$. By strong-normalization 3.14, $size(B') < size(A)$, so by well-founded induction, $reduce(\rightarrow_\beta)B' = \langle B, h_2 \rangle$. And so, $reduce(\rightarrow_\beta)A = \langle B, trans\ (single\ h')h_2 \rangle$.

✔ [Formalized: `Expr.reduce_complete`] □

**Theorem 3.24.** *Uniqueness of normal forms: if* $A, B, C \in E$ *and* $h_1 \vdash A \rightarrow_\beta^! B$ *and* $h_2 \vdash A \rightarrow_\beta^! C$*, then* $B = C$*.*

*Proof.* By completeness of $\beta$-reduction 3.23, $reduce(\rightarrow_\beta)A = \langle B, h_1 \rangle$ and $reduce(\rightarrow_\beta)A = \langle C, h_2 \rangle$, so $B = C$.

✔ [Formalized: `Expr.norm_unique`] □

Using our reduction function with soundness and completeness, we can decide $\beta$-equality.

**Definition 3.25.** $A =_\beta B$ iff there exists $C \in E$ such that $A \rightarrow_\beta^! C$ and $B \rightarrow_\beta^! C$ where $A, B \in E$.

**Theorem 3.26.** *If* $A, B \in E$*, then* $A =_\beta B$ *or* $A \neq_\beta B$*.*

*Proof.* Let $\langle A_1, h_1 \rangle \coloneqq reduce(\rightarrow_\beta)A$ and $\langle B_1, h_2 \rangle \coloneqq reduce(\rightarrow_\beta)B$. Then, $A_1$ and $B_1$ are normal forms by 3.22. We proceed by cases:

$A_1 = B_1$**:** If $A_1 = B_1$, then $A \rightarrow_\beta^! A_1$ and $B \rightarrow_\beta^! A_1$, so $A =_\beta B$.

$A_1 \neq B_1$**:** If $A_1 \neq B_1$, then $A \neq_\beta B$, since $A =_\beta B$ implies that there exists $C \in E$ such that $h_3 \vdash A \rightarrow_\beta^! C$ and $h_4 \vdash B \rightarrow_\beta^! C$. But, then $reduce(\rightarrow_\beta)A = \langle C, h_3 \rangle = \langle C, h_4 \rangle = reduce(\rightarrow_\beta)B$ (by propositional extensionality), in which case a contradiction is derived.

□

## 4. Inference Rules

DCC has decidable type-checking, dependent types, and uses affine logic. Every primitive object is well-typed, function types are quite legible, and explicit universe levels do not need to be specified (except in literal Sort $u$ terms). DCC achieves decidable type-checking by eagerly $\beta$-reducing all inferred types to canonical, normal form, and by defining definitional equality as $\beta$-equality. DCC achieves legible types and universe inference by encoding dependent type arguments as points of the respective combinator which index an applied environment argument, rather than by supplying type arguments as arguments themselves (resembling Martin Löf's notion of a Tarski universe [MS84, p. 88]).

We proceed by enumerating the inference rules for the core type formers, the rules for constant terms, and then the inference rules for all primitive combinators in the order they were defined in 3.

**Definition 4.1.** $A \equiv B$ iff $A =_\beta B$ where $A, B \in E$ 3.25.

**Definition 4.2.** Arrows, written in the meta-level as $A \to B$ are identified by the rule:

$$\textsc{Arrow Def.}\ \frac{A \to^!_\beta A' \qquad B \to^!_\beta B'}{A' \to B' \coloneqq ((\Pi\ (`\mathbb{1}\ A)\ (`\mathbb{1}\ (`A\ B)))*)}$$

We use $(\to\ \ A\ B)$ to explicitly denote the DCC expansion of $A \to B$.

**Definition 4.3.** We have the following mutually exclusive inference rules for $\Pi$:

$$\textsc{Pi}\ \frac{B : A \to \text{Sort } u \quad B \to^!_\beta B' \quad C : ((\Pi\ B_c\ C_c)x_C) \quad (B_c x_C) \to^!_\beta A \quad (C_C x_C) \to^!_\beta (\Pi\ B'\ (\Lambda\ A\ B'\ (`(\Sigma'\ A\ B')\ \text{Sort } v)))}{(\Pi\ B\ C) : A \to \text{Sort } \max(u, v)}$$

$$\textsc{Pi}\ \frac{B : (!\ A) \to \text{Sort } u \quad B \to^!_\beta B' \quad C : ((\Pi\ B_c\ C_c)x_C) \quad (B_c x_C) \to^!_\beta (!\ A) \quad (C_C x_C) \to^!_\beta (\Pi\ (!<>\ \ B')\ (!\Lambda\ A\ B'\ (`(!\ (\Sigma'\ A\ B'))\ \text{Sort } v)))}{(\Pi\ B\ C) : A \to \text{Sort } \max(u, v)}$$

$$\textsc{Pi}\ \frac{B : A \to \text{Sort } u \quad C : A \to A_2 \to \text{Sort } v}{(\Pi\ B\ C) : A \to \text{Sort } \max(u, v)}$$

$$\textsc{Pi}\ \frac{B : A \to \text{Sort } u \quad B \to^!_\beta B' \quad C : ((\Pi\ B_C\ C_C)x_C) \quad (B_c x_C) \to^!_\beta A \quad (C_C x_C) \to^!_\beta (\Pi\ B'\ (`A_2\ (`A_3\ \text{Sort } v)))}{(\Pi\ B\ C) : A \to \text{Sort } \max(u, v)}$$

We distinguish between four different kinds of indices $C : (x : A) \to (y : Bx) \to \text{Sort } v$, which are all arrows into Sort $v$ whose arguments may or may not depend on the context.

**Definition 4.4.** $\Sigma$-types are inferred as such:

$$\textsc{Sigma}\ \frac{A : \text{Sort } u \quad B : ((\Pi\ B\ C)env) \quad A \equiv (Benv) \quad (C\ env\ \text{hole}(h_{\max}(((\Pi\ B\ C)\ env)) + 1)) \to^!_\beta \text{Sort } v}{(\Sigma\ A\ B) : \text{Sort } \max(u, v)}$$

where $h_{\max} : E \to \mathbb{N}$ calculates the maximum "hole" in a tree and is used to test if $B$ is an arrow (but only in this rule).

**Definition 4.5.** Application is type-checked as such:

$$\textsc{App}\ \frac{f : ((\Pi\ B\ C)\ env) \quad x : A \quad (B\ env) \equiv A \quad (C\ env\ x) \to^!_\beta \tau}{(fx) : \tau}$$

Note that the inferred type of $(fx)$ is in normal form.

**Definition 4.6.** Constant terms are inferred like such:

| Sort | Top | Bot | Top intro | True | False |
|---|---|---|---|---|---|
| $\dfrac{}{\text{Sort } u : \text{Sort } u + 1}$ | $\dfrac{}{\mathbb{1} : \text{Sort } 0}$ | $\dfrac{}{\mathbb{0} : \text{Sort } 0}$ | $\dfrac{}{* : \mathbb{1}}$ | $\dfrac{}{\mathbb{1}_{\mathbb{2}} : \mathbb{2}}$ | $\dfrac{}{\mathbb{0}_{\mathbb{2}} : \mathbb{2}}$ |

**Definition 4.7.** The identity, quotation, and composition are inferred as such:

$$\textsc{Id}\ \frac{A : \text{Sort } u \qquad A \to^!_\beta A'}{(Id\ A) : A' \to A'} \qquad \textsc{Quot}\ \frac{x : A \qquad B : \text{Sort } v \qquad B \to^!_\beta B'}{(`B\ x) : B' \to A'}$$

$$\textsc{Comp}\ \frac{g : A \to B \qquad f : ((\Pi\ (`\mathbb{1}\ B)\ (`\mathbb{1}\ C))*) \qquad g \to^!_\beta g'}{(\circ\ f\ g) : ((\Pi\ (`\mathbb{1}\ A)\ (`\mathbb{1}\ (\circ\ C\ g)))*)}$$

**Example 4.8.** $((Id\ \mathbb{1})*) : \mathbb{1}$.

This is since $\mathbb{1} : \text{Sort } 0$ implies $(Id\ \mathbb{1}) : \mathbb{1} \to \mathbb{1}$, and $\mathbb{1} \to \mathbb{1} = ((\Pi\ (`\mathbb{1}\ \mathbb{1})\ (`\mathbb{1}\ (`\mathbb{1}\ \mathbb{1})))*)$ and $* : \mathbb{1}$ and $((`\mathbb{1}\ \mathbb{1})*) \equiv \mathbb{1}$, and $((`\mathbb{1}\ \mathbb{1})*) \to^!_\beta \mathbb{1}$.

In the familiar $(x : A) \to Bx$ notation, the inference rule for composition is written as:

$$\textsc{Comp}\ \frac{g : A \to B \qquad f : (x : B) \to Cx \qquad g \to^!_\beta g'}{(\circ\ f\ g) : (x : A) \to C(g'x)}$$

Similarly, the inference rule for $\Pi$ may be written notationally as:

$$\textsc{Pi}\ \frac{B : A \to \text{Sort } u \qquad C : (x : A) \to (y : Bx) \to \text{Sort } v}{(\Pi\ B\ C) : A \to \text{Sort } \max(u, v)}$$

From henceforth, we will use the notation $(x : A) \to Bx$ to denote dependent products for clarity. Consult the Appendix A for the expanded inference rules.

**Definition 4.9.** Projections and currying are as such:

$$\textsc{Fst}\ \frac{B : A \to \text{Sort } v \qquad B \to^!_\beta B'}{(Fst\ B) : (\Sigma\ A\ B') \to A} \qquad \textsc{Snd}\ \frac{B : A \to \text{Sort } v \qquad B \to^!_\beta B'}{(Snd\ B) : (x : (\Sigma\ A\ B')) \to ((\circ\ B'\ (Fst\ B'))x)}$$

$$\textsc{Curry}\ \frac{A : \text{Sort } u \quad A \to^!_\beta A' \qquad B : A' \to \text{Sort } v \qquad B \to^!_\beta B' \qquad x : (x : (\Sigma\ A'\ B')) \to Cx}{(\Lambda\ A\ B\ x) : (x : A') \to (y : B'x) \to ((\Lambda\ A'\ B'\ C)xy)}$$

**Definition 4.10.** Evaluation is as such:

$$\textsc{Eval}\ \frac{A : \text{Sort } u \quad A \to^!_\beta A' \quad B : A' \to \text{Sort } v \quad B \to^!_\beta B' \quad x : (x : A') \to (y : B'x) \to Cxy}{(Eval\ A\ B\ x) : (x : (\Sigma\ A'\ B')) \to ((Eval\ A'\ B'\ C)x)}$$

**Definition 4.11.** Exchange is as such:

$$\textsc{Swap}\ \frac{A : \text{Sort } u \quad B : \text{Sort } v \quad A \to^!_\beta A' \quad B \to^!_\beta B'}{(Swap\ A\ B) : (\Sigma\ A'\ (`A'\ B')) \to (\Sigma\ B'\ (`B'\ A'))}$$

**Definition 4.12.** The basic operations on boxed terms are as such:

$$\textsc{Unbox}\ \frac{A : \text{Sort } u \quad A \to^!_\beta A'}{(*\ A) : (!\ A') \to A'} \qquad \textsc{Map}\ \frac{x : A \to \text{Sort } v}{(!<>\ \ x) : (!\ A) \to \text{Sort } v}$$

$$\textsc{Fmap}\ \frac{x : (x : A) \to Bx}{(\$<>\ \ x) : (x : (!\ A)) \to ((!<>\ \ B)x)}$$

**Definition 4.13.** Currying on boxes is as such:

$$\textsc{Curry'}\ \frac{A : \text{Sort } u \quad A \to^!_\beta A' \quad B : A' \to \text{Sort } v \quad B \to^!_\beta B' \quad x : (x : (!\ (\Sigma\ A'\ B'))) \to Cx}{(!\Lambda\ A\ B\ x) : (x : (!\ A')) \to (y : ((!<>\ \ B')x)) \to ((!\Lambda\ A'\ B'\ C)xy)}$$

**Definition 4.14.** Pairs are as such:

$$\textsc{Pair}\ \frac{C \to^!_\beta C' \qquad x : (x : A) \to Bx \qquad y : (x : A) \to (y : Bx) \to C'xy \qquad C : (x : A) \to (y : Bx) \to \text{Sort } s}{(\langle,\rangle C\ x\ y) : (x : (!\ A)) \to ((\Sigma'\ B\ C')x)}$$

**Definition 4.15.** Equality formers are as such:

$$\textsc{Eq'}\ \frac{B : A \to \text{Sort } v \quad B \to^!_\beta B'}{(='\ \ B) : (x : A) \to (y : B'x) \to (z : B'x) \to \text{Sort } 0}$$

$$\textsc{Eql}\ \frac{B : A \to \text{Sort } v \quad B \to^!_\beta B'}{(Eql\ B) : (x : (!\ A)) \to (y : ((!<>\ \ B')x)) \to \text{Sort } 0}$$

**Definition 4.16.** The affine $\Sigma$-former is as such:

$$\textsc{Sigma'}\ \frac{B : A \to \text{Sort } v \quad B \to^!_\beta B' \quad C : (x : A) \to (y : B'x) \to \text{Sort } s}{(\Sigma'\ B\ C) : (x : (!\ A)) \to \text{Sort } \max(v, s)}$$

**Definition 4.17.** The recursors for $\mathbb{2}$ and $\mathbb{0}$ are as such:

$$\textsc{Bool Rec}\ \frac{B : A \to \mathbb{2} \to \text{Sort } v \quad B \to^!_\beta B' \qquad x : (x : A) \to (y : \mathbb{2}) \to B'xy \qquad y : (x : A) \to (y : \mathbb{2}) \to B'xy}{(\mathbb{2}.Rec\ x\ y) : (x : A) \to (y : \mathbb{2}) \to B'xy}$$

$$\textsc{Bot Rec}\ \frac{B : A \to \mathbb{0} \to \text{Sort } v \quad B \to^!_\beta B'}{(\mathbb{0}.Rec\ B) : (x : A) \to (y : \mathbb{0}) \to B'xy}$$

**Definition 4.18.** $\Sigma$-couples are as such:

$$\textsc{Couple}\ \frac{x : A \quad B : A \to \text{Sort } v \quad B \to^!_\beta B' \qquad (B'x) \to^!_\beta \tau_2 \quad y : \tau_2}{(,B\ x\ y) : (\Sigma\ A\ B')}$$

**Definition 4.19.** Finally, affine terms are as such:

$$\textsc{Bang}\ \frac{A : \text{Sort } u}{(!\ A) : \text{Sort } u} \qquad \textsc{Box}\ \frac{x : A}{(\&\ x) : (!\ A)}$$

4.1. **Type-Checking is Decidable.** Our inference rules do not assume the existence of a catch-all conversion rule. Instead, inferred types are reduced to normal form in each rule individually, and definitional equality of terms is checked by comparing their normal-form reducts given by *reduce*. This enables decidable type-checking. We have implemented a type-checker on a deep embedding of DCC in Lean 4, and have shown that it to be judgmentally complete, making type-checking decidable.

The source code of the type-checker is available in the Appendix A. Of note, judgments carry computational content and live in **Type**, rather than **Prop**: Judge : $E \to E \to Type$. The type-checker is total and equipped with soundness: $infer : (e : E) \to$ Except (TypeError $e$)$(\Sigma'(\tau : E), \text{Judge } e\ \tau)$. Judgmental completeness is provable from the fact that all the inference rules are mutually exclusive, and there is exactly one canonical type for a given term since catch-all conversion is not permitted.

**Theorem 4.20.** *Judgmental completeness: If $e, \tau \in E$ and $h \vdash e : \tau$, then $infer\, e = ok\langle \tau, h\rangle$.*

*Proof.* By induction on the typing derivation $h$. Cases are mutually exclusive and we have soundness, so the proof is reduced to checking that each branch implies that its corresponding branch in $infer$ is $ok$. In each case, if a definitional equality is supposed by $h$, such that $A \equiv B$ for some $A, B \in E$ implies $e : \tau$, then there exists $C \in E$ such that $h_{eq,1} \vdash A \to^!_\beta C$ and $h_{eq,2} \vdash B \to^!_\beta C$. So, by completeness of $\beta$-reduction with respect to normal forms 3.23, $reduce(\to_\beta)A = \langle C, h_{eq,1}\rangle = \langle C, h_{eq,2}\rangle = reduce(\to_\beta)B$. Hence, the corresponding equality check in $infer\ e$ passes, and so in each case, $infer\ e = ok\ \langle \tau, h\rangle$.
✔ [Formalized: `Expr.Judge.complete`] □

**Theorem 4.21.** *If $A, \tau_1, \tau_2 \in E$ and $e : \tau_1$ and $e : \tau_2$, then $\tau_1 = \tau_2$.*

*Proof.* If $h_1 \vdash e : \tau_1$ and $h_2 \vdash e : \tau_2$, then by judgmental completeness 4.20, $infer\ e = ok\ \langle \tau_1, h_1\rangle = infer\ e = ok\ \langle \tau_2, h_2\rangle$, so $\tau_1 = \tau_2$.
✔ [Formalized: `Expr.Judge.unique`] □

**Theorem 4.22.** *Inferred types are normal: If $e, \tau \in E$ and $h \vdash e : \tau$, then $\tau$ is a $\beta$-normal form.*

*Proof.* We proceed by induction on the term $e$:

- $e$ **is a constant:** If $e \in E_c$, then $\tau$ is one of $\{\text{Sort } u + 1, \text{Sort } 0, \mathbb{0}, \mathbb{1}, \mathbb{2}\}$, in which case $\tau$ is $\beta$-normal.
- $e = (c\ x_0 \dots x_n)$**, where** $c \in C$ **and** $\{x_0 \dots x_n\} \subset E$**:** If $e$ is a combinator, then any point mentioned in its type $\tau$ has been reduced to $\beta$-normal form, or was inferred from the type of a point, in which case it is in $\beta$-normal form by the inductive hypothesis. And, if $e$ is a combinator, it is either a function, a type former, or a boxed term:
  - $e$ **is a function:** Then $\tau$ is of the form $((\Pi\ B\ C)x)$, where $B$, $C$, and $x$ are in $\beta$-normal form due to the above reasoning.
  - $e$ **is a type former:** Then $\tau$ is of the form Sort $u$, in which case it is in $\beta$-normal form.

$e$ **is a boxed term:** Then $\tau$ is of the form $(!\ A)$, where $A$ is inferred by a point of $e$, in which case $A$ is in $\beta$-normal form by the inductive hypothesis. Hence, $\tau$ is in $\beta$-normal form.

$e = (fx)$**, where** $f \in E$ **and** $x \in E$**:** If $e$ is an application, then $f : ((\Pi\ B\ C)\ env)$ and $x : A$ where $(B\ env) \equiv A$. So, $\tau$ is the $\beta$-normal reduct of $C\ env\ x$ (per 4.5), and it is in $\beta$-normal form.

✔ [Formalized: `Expr.Judge.is_norm`] □

4.2. **Subject-Reduction, Progress, and Consistency.** The subject-reduction and progress theorems hold in DCC. We show that if $A : \tau$ and $A \to_\beta B$, then $B : \tau$. We define a set of canonical values Val $\subset E$ syntactically and show that if $A : \tau$, then there must exist $B$ such that $A \to_\beta B$ or $A \in$ Val. Using these results, we show that $\mathbb{0}$ is uninhabited.

**Theorem 4.23.** *Preservation: If $A, B, \tau \in E$, $ht \vdash A : \tau$, and $hr \vdash A \to_\beta B$, then $B : \tau$.*

*Proof.* We proceed by induction on $A$:

$A$ **is of the form** $(fx)$ **and** $B$ **is of the form** $(f'x)$ **where** $hr_f \vdash f \to_\beta f'$**:** Then $f : ((\Pi\ B\ C)\ env)$ and $x : A$ such that $(B\ env) \equiv A$. Hence, the inductive hypothesis via $hr_f$ that $f' : ((\Pi\ B\ C)\ env)$. So, by the inference rule for application, $(f'x) : \tau$.

$A$ **is of the form** $(fx)$ **and** $B$ **is of the form** $(fx')$ **where** $hr_x \vdash x \to_\beta x'$**:** Then $f : ((\Pi\ B\ C)\ env)$ and $x : A$ such that $(Benv) \equiv A$ and $C\ env\ x \to^!_\beta \tau$. Hence, the inductive hypothesis via $hr_x$ that $x' : A$, and by confluence and argument congruence, $(C\ env\ x') \to^!_\beta \tau$, so $(fx') : \tau$.

$A$ **is of the form** $(c\ x_0 \dots x_n)$ **where** $c \in C$ **and** $\{x_0 \dots x_n\} \subset E$**:** If $(c\, x_0 \dots x_n) : \tau$, then each point $x_i$ for all $i < n$ is asserted by $ht$ to have a type $\tau_i$ and may appear as a sub-expression of $\tau$. And, since $(c\, x_0 \dots x_n) \to_\beta B$, $B$ must be of the form $(c\, x'_0 \dots x'_n)$ where $x_j \to_\beta x'_j$ for some $j < n$. So, by the inductive hypothesis, $x'_j : \tau_j$. Therefore, $(c\ x'_0 \dots x'_n) : \tau$ by the inference rule for the combinator $c$.

$A$ **is of the form** $(fx)$ **and** $hr$ **is given by a head step:** In case $hr$ is given by a head reduction step, we proceed by cases. In each head-reduction case, the typing relation is inverted until we reach the leftmost combinator $ht_1 \vdash (c\ x_0 \dots x_n) : \tau_1$ which is a sub-expression of $f$. We invert $ht_1$ and destruct all available definitional equalities and reductions to $\beta$-normal form. Then, we reconstruct the judgment $B : \tau$ from the available equalities. See the formalized lean proof for an exhaustive technical presentation of this technique.

✔ [Formalized: `Expr.Judge.preservation`] □

**Definition 4.24.** The set of canonical values is defined to be the union of the set of constant terms $E_c$, the set of combinators whose points are values $\{(c\ x_0 \dots x_n) \mid c \in C, \{x_0 \dots x_n\} \subset \text{Val}\}$, application of the reflexivity combinator, and all possible partially applied terms:

$$E_c \subset \text{Val}$$
$$\{(c\ x_0 \dots x_n) \mid c \in C, \{x_0 \dots x_n\} \subset \text{Val}\} \subset \text{Val}$$

$$\{((\Pi\ B\ C)x) \mid B, C, x \in \text{Val}\} \subset \text{Val}$$
$$\{((!\Lambda\ A\ B\ x)(\&\ y)) \mid A, B, x, y \in \text{Val}\} \subset \text{Val}$$
$$\{((\Lambda\ A\ B\ x)y) \mid A, B, x, y \in \text{Val}\} \subset \text{Val}$$
$$\{((='\ B)xy) \mid B, x, y \in \text{Val}\} \subset \text{Val}$$

$$\{((refl\ B)(\&\ x)) \mid B, x \in \text{Val}\} \subset \text{Val} \qquad \{((Eql\ B)(\&\ x)) \mid B, x \in \text{Val}\} \subset \text{Val}$$
$$\{((refl\ B)(\&\ x)(\&\ y)) \mid B, x, y \in \text{Val}\} \subset \text{Val} \qquad \{((='\ B)x) \mid B, x \in \text{Val}\} \subset \text{Val}$$
$$\{((\mathbb{2}.Rec\ B\ x\ y)z) \mid B, x, y, z \in \text{Val}\} \subset \text{Val} \qquad \{((\mathbb{0}.Rec\ B)x) \mid B, x \in \text{Val}\} \subset \text{Val}$$

Note that $((\mathbb{0}.Rec\ B)x)$ denotes partial application of the recursor for $\mathbb{0}$ with an environment, not an inhabitant of $\mathbb{0}$.

**Lemma 4.25.** *If $h \vdash (fx) \in$ Val, then $f \in$ Val.*

*Proof.* By case analysis on $h$.
✔ [Formalized: `Expr.Value.congrFun`] □

**Lemma 4.26.** *If $h \vdash (fx) \in$ Val, then $x \in$ Val.*

*Proof.* By case analysis on $h$.
✔ [Formalized: `Expr.Value.congrArg`] □

**Lemma 4.27.** *If $h \vdash e \in$ Val, then $e$ is a $\beta$-normal form.*

*Proof.* By induction on $h$.
✔ [Formalized: `Expr.Value.is_norm`] □

Progress may be established by noticing that the only circumstance where a reduction gets "stuck" is when a function is applied the appropriate number of arguments, but they are all in $\beta$-normal form but are not the appropriate shape (i.e., $(\&\ x)$, $(,B\ x\ y)$, $\mathbb{1}_2$, or $\mathbb{0}_2$). By inductive reasoning, an argument $x$ to a function in the term $(fx)$ must either be a value, or must make a step. And, since the leftmost combinator $(c\ cx_0 \ldots cx_n)$ which is a subexpression of $f$ has type $(x_0 : \tau_0) \to (x_1 : \tau_0 x) \to \ldots$, then $x_0 : \tau_0$ and $x_{i+1} : (\tau_{i+1}\ x_0 \ldots x_i)$ for all arguments $i < n$. It suffices to show that this fact implies $x_i$ has the expected shape of an inhabitant of $\tau_i \ldots$, in which case, $(fx)$ is a value by the appropriate rule. We do so by proving inversion lemmas relating judgments to the shapes of their expected inhabitants.

**Lemma 4.28.** *If $x \in$ Val and $h \vdash x : (!\ A)$, then there exists $x_0 \in E$ such that $x = (\&\ x_0)$.*

*Proof.* We proceed by case analysis on the value $x$. In any case where $x$ is not of the form $(\&\ x_0)$, the judgment $h$ may be inverted to show that $x : \tau$ and $\tau$ is not of the form $(!\ A)$. For example, combinators which are functions never have a type of the form $(!\ A)$. For the type-formers, $x : \text{Sort}\ u$ for some $u \in \mathbb{N}$, but $\text{Sort}\ u \neq (\ A)$ for all $A \in E$. In the case of the canonical partially applied combinators, $h$ is similarly inverted to show that $x : \tau$ and $\tau$ is of the form $((\Pi\ B\ C)\ env)$, but $((\Pi\ B\ C)\ env) \neq (!\ A)$ for all $A \in E$.
✔ [Formalized: `Expr.Value.inv_box`] □

**Lemma 4.29.** *If $h \vdash ((\Pi\ B\ C)x) : \tau$, then there exists $u \in \mathbb{N}$ such that $\tau = Sort\ u$.*

*Proof.* By inversion on $h$.

✔ [Formalized: `Expr.Judge.inv_pi`] □

**Lemma 4.30.** *If $h \vdash ((Eql\ B)(\&\ x)(\&\ y)) : \tau$, then $\tau = Sort\ 0$.*

*Proof.* By inversion on $h$.
✔ [Formalized: `Expr.Judge.inv_eql`] □

**Lemma 4.31.** *If $h \vdash ((refl\ B)(\&\ x)(\&\ y)) : \tau$, and $(Bx) \to_\beta^! B'$ and $y \to_\beta^! y'$, then $\tau = ((=\ B')y'y')$.*

*Proof.* By inversion on $h$.
✔ [Formalized: `Expr.Judge.inv_refl`] □

**Lemma 4.32.** *If $ht \vdash e : \mathbb{0}$ and $hv \vdash e \in Val$, then $\bot$.*

*Proof.* We proceed by case analysis on $ht$. The proof can be reduced to checking that no partial application of a combinator can inhabit $\mathbb{0}$, since no inference rule could yield $\mathbb{0}$ in the consequent besides application. In case $e = (fx)$ where $f, x \in E$, we observe that $(fx)$ is a value, so it must be a partial application of a function, or a full application of $refl$. In case $f$ is a function value whose arity is greater than one, inversion shows that there must exist $B, C, env \in E$ such that $(fx) : ((\Pi\ B\ C)\ env)$, but $((\Pi\ B\ C)\ env) \neq \mathbb{0}$ for all $B, C, env \in E$. In case $(fx)$ is a full application of $refl$, we know by 4.31 that there must exist $B, y \in E$ such that $(fx) : ((=\ B)yy)$, but $((=\ B)yy) \neq \mathbb{0}$ for all $B, y \in E$, so $\neg e : \mathbb{0}$ and thus a contradiction is derived.
✔ [Formalized: `Expr.Judge.elim_bot_value`] □

**Lemma 4.33.** *If $ht \vdash e : (\Sigma\ A\ B)$ and $hv \vdash e \in Val$, then there exist $x_0, y_0 \in E$ such that $e = (,B\ x\ y)$.*

*Proof.* By case analysis on $hv$ and parallel inversion on $ht$.
✔ [Formalized: `Expr.Value.inv_sigma`] □

**Lemma 4.34.** *If $ht \vdash e : \mathbb{2}$ and $hv \vdash e \in Val$, then $e = \mathbb{1}_\mathbb{2}$ or $e = \mathbb{0}_\mathbb{2}$.*

*Proof.* By case analysis on $hv$ and parallel inversion on $ht$.
✔ [Formalized: `Expr.Value.inv_bool`] □

Using these facts about inversion, we prove progress.

**Theorem 4.35.** *Progress: If $h \vdash e : \tau$, then $e \in Val$ or there exists $e' \in E$ such that $e \to_\beta e'$.*

*Proof.* The proof proceeds by induction on $E$. If $e \in E_c$, then $e \in$ Val. If $e = (c\ x_0 \dots x_n)$ where $c \in C$ and $\{x_0 \dots x_n\} \subset E$, then $h$ demonstrates that there exist $\tau_i$ such that $x_i : \tau_i$ for all $i < n$, and so each point of $e$ is either a value, or can make a step. In case all points of $e$ are values, $e \in$ Val. If any point can make a step, then so can $e$ by the point rule. If $e$ is of the form $(fx)$ where $f, x \in E$, then $h$ gives that $f : ((\Pi\ B\ C)\ env)$ for some $B, C, env \in E$, and $x : A$ and $(B\ env) \equiv A$. Furthermore, by the inductive hypothesis, both $f$ and $x$ must be values or able to make a step. In case either $f$ or $x$ can make a step, function congruence and argument congruent respectively give that $e$ can make a step. In case both $f$ and $x$ are values, we perform case analysis on $f$, and show that $(fx)$ is either a partial application of a combinator, a full application of $refl$, or, by the established inversion lemmas, $x$ is of the right shape such that $(fx)$ can make a head-step reduction.
✔ [Formalized: `Expr.Judge.progress`] □

**Theorem 4.36.** *Consistency: There does not exist $e \in E$ such that $e : \mathbb{0}$.*

*Proof.* This fact is due to the inversion lemma on values inhabiting $\mathbb{0}$ 4.32, well-founded induction by decreasing size under $\rightarrow_\beta$, progress, and preservation.

Suppose $e \in E$ such that $h \vdash e : \mathbb{0}$. Then, by progress 4.35, $e \in \text{Val}$ or there exists $e' \in E$ such that $e \rightarrow_\beta e'$. If $e \in \text{Val}$, then by the bottom type value inhabitance inversion lemma (4.32), a contradiction is derived. If $e \rightarrow_\beta e'$, then $size(e') < size(e)$ by strong normalization 3.14. And, by preservation 4.23, $e' : \mathbb{0}$. So, by well-founded induction, a contradiction is derived.

✔ [Formalized: `Expr.Judge.elim_bot`] □

We note that the subject-reduction, progress, and consistency are proven as computable definitions, rather than theorems in **Prop**. Furthermore, the progress theorem gives a computable choice of $e' \in E$ such that $e \rightarrow_\beta e'$ in case $e$ is not a value, rather than merely proving its existence existentially.

## 5. Computable Equality Reflection

The extensional equality reflection theorem holds in DCC. This is a result of decidable type-checking and strong normalization avoiding a generalized conversion constructor. Since inferred types are always in $\beta$-normal form, and since $refl$ is the canonical inhabitant of equality, $e : (= A\ x\ y)$ iff $x \equiv y$. This fact is provable from progress, preservation, and well-founded recursion by strong normalization.

**Lemma 5.1.** *If $ht \vdash e : (= A\ x\ y)$ and $hv \vdash e \in \text{Val}$, then $x \equiv y$.*

*Proof.* The proof proceeds by case analysis on the typing derivation $ht$. No inference rule can yield $(= A\ x\ y)$ in the consequent aside from application. Hence, $e$ must be of the form $(fx)$ where $f, x \in E$ and $f, x \in \text{Val}$. We note that $(= A\ x\ y)$ is not a function type. Hence, $(fx)$ cannot be a partial application of a function, but $(fx)$ is not a term with points, nor is it a constant value. So, $(fx)$ must be of the form $((refl\ B)(\&\ x)(\&\ y))$. We demonstrate this fact by inverting the typing derivation $ht$ in each case where $f$ is a value. We demonstrate that in every case except where $(fx) = ((refl\ B)(\&\ x)(\&\ y))$, $f$ is a function whose arity is greater than one, and so $(fx)$ must be typed at $\tau$ where $\tau$ is of the form $((\Pi\ B\ C)\ env)$ where $B, C, env \in E$, but $((\Pi\ B\ C)\ env) \neq (= A\ x\ y)$ for all $B, C, env, A, x, y \in E$.

So, if $(fx) = ((refl\ B)(\&\ x)(\&\ y))$ for some $B, x, y \in E$, then by inversion on $refl$ values 4.31, $((refl\ B)(\&\ x)(\&\ y))$ must be typed at $\tau$ where $(Bx) \rightarrow_\beta^! B'$ and $y \rightarrow_\beta^! y'$, and $\tau = ((=\ B')y'y')$. But by uniqueness of judgments 4.21, $(=\ B'\ y'\ y') = (=\ A\ x\ y)$, so $y' = x$ and $y' = y$. Hence, $x \equiv y$ by reflexivity.

✔ [Formalized: `Expr.Value.elim_eq`] □

**Theorem 5.2.** *Equality reflection: If $e : (= A\ x\ y)$, then $x \equiv y$.*

*Proof.* This fact is provable by progress, preservation, and well-founded induction with respect to the reduct of $e$, in case it is not a value. By progress 4.35, $e \in \text{Val}$ or there exists $e' \in E$ such that $e \rightarrow_\beta e'$. If $e \in \text{Val}$, then by the above 5.1, $x \equiv y$. If $e \rightarrow_\beta e'$, then $size(e') < size(e)$ by strong normalization 3.14. And, by preservation 4.23, $e' : (= A\ x\ y)$. So, by well-founded induction, $x \equiv y$.

✔ [Formalized: `Expr.Judge.elim_eq`] □

We note that equality reflection is a computable definition in DCC, rather than a theorem in **Prop**. That is, it has computational content and can be used outside of proofs.

## 6. Compiling Lean 4 to DCC

The subset of Lean 4 that maps to the base Calculus of Constructions without equality can be compiled to DCC. We note that working examples are somewhat limited, since we refrained from equipping DCC with $\eta$-reduction, as it is outside of the scope of this paper and not necessary to demonstrate the fundamental theorems. However, in DCC with $\eta$-reduction, the dependent $K$ combinator written as a Lean term can be compiled to DCC. We do not propose a method for encoding inductive types in DCC, as it is out of scope.

Compilation is quite straightforward. Contexts introduced by binders during compilation are represented as $\Sigma$ or bang ! terms and the program is always initialized at the top-level with the empty context (! $\mathbb{1}$). Debruijn-indexed variables are compiled to composed projections, $\lambda$-binders are compiled to !$\Lambda$-terms, and $\forall$-terms are compiled to $\Pi$-terms. The only complication is DCC's affine restriction: Lean terms are preprocessed to annotate code denoting how many times a resource should be banged, and values are boxed appropriately.

We are most concerned with compiling the subset of Lean with $\forall$, $\lambda$, sorts, booleans, top and bottom types (and their corresponding introduction rules), $\Sigma$-types, and $\Sigma$-terms.

**Definition 6.1.** We denote $L_c$ as the set of compilation candidate constant Lean terms (where $\top$ denotes the Lean term $True$, $\bot$ denotes the Lean term $False$, and $*$ denotes the Lean term $True.intro$, $\mathbb{2}$ denotes $Bool$, and $\mathbb{1}_\mathbb{2}$ denotes $Bool.true$ and $\mathbb{0}_\mathbb{2}$ denotes $Bool.false$):

$$L_c \coloneqq \{\top, *, \bot, \mathbb{2}, \mathbb{1}_\mathbb{2}, \mathbb{0}_\mathbb{2}, \text{Sort } u\}$$

where $u \in \mathbb{N}$ (that is, we only consider concrete universes and not metavariables).

**Definition 6.2.** We denote $L$ as the set of candidate Lean terms:

$$L_c \subset L \quad \{n \in \mathbb{N} \mid n \text{ is a variable}\} \subset L$$
$$\{(MN) \mid M, N \in L\} \subset L \quad \{\lambda x : A.M \mid A, M \in L\} \subset L$$
$$\{\forall x : A.B \mid A, B \in L\} \subset L \quad \left\{\sum_{x:A} B(x) \mid A, B \in L\right\} \subset L$$
$$\{\text{Shared } A \mid A \in L\} \subset L \quad \{\text{share } Ax \mid A, x \in L\} \subset L$$

$$\{\langle x, y\rangle \mid x, y \in L\} \subset L$$

where Shared is a marker denoting a bang-type and share is a marker denoting a box-value.

**Definition 6.3.** We write the set of contexts tracked in compilation and introduced by binders as Ctx:

$$\mathbb{1} \in \text{Ctx} \qquad \{(\Sigma\ A\ B) \in E \mid A \in \text{Ctx}, B \in \text{Val}\} \subset \text{Ctx} \qquad \{(!\ A) \in E \mid A \in \text{Ctx}\} \subset \text{Ctx}$$

Compilation is distinct at the top level and under binders ($\lambda$, $\forall$, and $\Sigma$-types and terms). We write $[\![M]\!]_\Gamma \in E$ to denote compilation of the term $M \in L$ under a binder (abstraction). We write $[\![M]\!] \in E$ to denote compilation of the term $M \in L$ at the top-level when it is not under a binder. Variables are abstracted by composed projection and unboxing of banged contexts, or the identity if the context is empty. Constants are abstracted by quoting the context, or mapping the context with ! <> if the constant is shared.

**Definition 6.4.** Variable abstraction is as such:

$$\frac{}{[\![n]\!]_{\mathbb{1}} \coloneqq (Id\ \mathbb{1})} \qquad \frac{[\![n]\!]_\Gamma = n'}{[\![n]\!]_{(!\ \Gamma)} \coloneqq (\circ\ n'\ (*\ \Gamma))}$$

$$\frac{}{[\![0]\!]_\Gamma \coloneqq (Snd\ \Gamma)} \qquad \frac{[\![n]\!]_\Gamma = n'}{[\![n+1]\!]_{(\Sigma\ \Gamma\ \Delta)} \coloneqq (\circ\ n'\ (Fst\ \Delta))}$$

**Definition 6.5.** Constant abstraction is as such:

$$\frac{}{[\![\top]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{1})} \qquad \frac{}{[\![*]\!]_\Gamma \coloneqq (`\Gamma\ *)}$$

$$\frac{}{[\![\bot]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{0})} \qquad \frac{}{[\![\mathbb{2}]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{2})}$$

$$\frac{}{[\![\mathbb{1}_{\mathbb{2}}]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{1}_{\mathbb{2}})} \qquad \frac{}{[\![\mathbb{0}_{\mathbb{2}}]\!]_\Gamma \coloneqq (`\Gamma\ \mathbb{0}_{\mathbb{2}})}$$

$$\frac{}{[\![\text{Sort } u]\!]_\Gamma \coloneqq (`\Gamma\ \text{Sort } u)}$$

$\lambda$-terms are always abstracted with $!\Lambda$, since we assume the context is always of the form $(!\ \Gamma) \in \text{Ctx}$. $\forall$-abstraction distinguishes between $\forall$-types which depend on the context and those which do not (i.e., if $A$ or $B$ does not depend on the context in $\forall A, B$). In case a $\forall$-type is nondependent, it is abstracted as a constant arrow discarding the context as in 4.2. $\Sigma$-terms also assume the context is shared, since $A$ and $B$ may depend on the context in $\sum_{x:A} B(x)$ and are compiled to a $\Sigma'$ type former which duplicates the context across the abstractions of $A$ and $B$.

**Definition 6.6.** Binder abstraction is as such:

$$\frac{\Gamma \text{ is not banged} \quad [\![A]\!]_\Gamma = A' \quad [\![M]\!]_{(\Sigma\ \Gamma\ A')} = M'}{[\![\lambda x : A.M]\!]_\Gamma \coloneqq (\Lambda\ \Gamma\ A'\ M')}$$

$$\frac{[\![A]\!]_\Gamma = A' \quad [\![M]\!]_{(!\ (\Sigma\ \Gamma\ A'))} = M'}{[\![\lambda x : (\text{Shared } A).M]\!]_{(!\ \Gamma)} \coloneqq (!\Lambda\ \Gamma\ A'\ M')}$$

$$\frac{\Gamma \text{ is not banged} \quad A \text{ or } B \text{ contain free variables} \quad [\![A]\!]_\Gamma = A' \quad [\![B]\!]_{(\Sigma\ \Gamma\ A')} = B'}{[\![\forall x : A, B]\!]_\Gamma \coloneqq (\Pi\ A'\ (\Lambda\ \Gamma\ A'\ B'))}$$

$$\frac{A \text{ or } B \text{ contain free variables} \quad [\![A]\!]_{(!\ \Gamma)} = A' \quad [\![B]\!]_{(!\ (\Sigma\ \Gamma\ A'))} = B'}{[\![\forall x : A, B]\!]_{(!\ \Gamma)} \coloneqq (\Pi\ A'\ (!\Lambda\ \Gamma\ A'\ B'))}$$

$$\frac{\text{neither } A \text{ nor } B \text{ contain free variables} \quad [\![A]\!]_{(!\ \mathbb{1})} = A' \quad [\![B]\!]_{(!\ \mathbb{1})} = B'}{[\![\forall x : A, B]\!]_\Gamma \coloneqq (`\Gamma\ (\to\ A'\ B'))}$$

$$\frac{[\![A]\!]_\Gamma = A' \quad [\![B]\!]_{(\Sigma\ \Gamma\ A')} = B'}{\left[\!\!\left[\sum_{x:A} B(x)\right]\!\!\right]_{(!\ \Gamma)} \coloneqq (\Sigma'\ A'\ B')}$$

where $\to$ denotes the DCC arrow expansion as in 4.2.

Applications and $\Sigma$-terms under binders duplicate the context across their constituent components ($M$ and $N$ in $(MN) \in L$ and $x$ and $y$ in $\langle x, y\rangle \in L$). So, they are abstracted with the pair combinator 4.14, assuming the context is of the form $(!\ \Gamma) \in \mathrm{Ctx}$. Applications $(MN) \in L$ are abstracted by composing the evaluation combinator on $M'$ with duplication of the context $x : \Gamma$ via the pair combinator to yield $(Eval\ A\ B\ M')((, B_2\ x\ (N'x))) \to_\beta (M'x(N'x))$.

**Definition 6.7.** Values are abstracted as such:

$$\frac{\begin{array}{c}[\![x]\!]_\Gamma = x', x' : ((\Pi\ B_1\ C_1)x_1) \quad [\![y]\!]_\Gamma = y', y' : ((\Pi\ B_1\ C_1)x_1) \\ (\circ\ y'\ (Fst\ (C_1x_1))) : (x : A) \to (y : Bx) \to Cxy\end{array}}{[\![\langle x, y\rangle_{(!\ \Gamma)} \coloneqq (\langle,\rangle C_3\ x'\ (\Lambda\ (B_1x_1)\ (C_1x_1)\ (\circ\ y'\ (Fst\ (C_1x_1)))))]\!]}$$

$$\frac{\begin{array}{c}[\![M]\!]_\Gamma = M', M' : ((\Pi\ B_1\ C_1)x_1) \quad [\![N]\!]_\Gamma = N', N' : ((\Pi\ B_2\ C_2)x_2) \\ (\circ\ N'\ (Fst\ (`\Gamma\ \Gamma))) : (x : A) \to (y : Bx) \to Cxy\end{array}}{[\![(MN)]\!]_{(!\ \Gamma)} \coloneqq (\circ\ (Eval\ (B_1x_1)\ (C_2x_2)\ M')\ (\Lambda\ \Gamma\ (`\Gamma\ \Gamma)\ (\circ\ N'\ (Fst\ (`\Gamma\ \Gamma)))))}$$

$$\frac{[\![x]\!]_\Gamma \coloneqq x'}{[\![\text{Shared } A]\!]_{(!\ \Gamma)} \coloneqq (!<>\ x')} \qquad \frac{[\![x]\!]_\Gamma \coloneqq x'}{[\![\text{share } Ax]\!]_{(!\ \Gamma)} \coloneqq (\$<>\ x')}$$

Compilation at the top-level differs from abstraction in that the context is empty. So, all top-level binders' binder types are compiled recursively without a context. Also, the topmost $\lambda$-binder is always applied the empty context $(\&\ *) : (!\ \mathbb{1})$. Finally, all user-supplied top-level values are automatically shared based on the computed maximum context duplication depth (this is done as a pre-processing step by marking values with *share*).

**Definition 6.8.** Constants are compiled as in 6.5, except the produced DCC terms are not quoted:

$$\frac{}{[\![\top]\!] \coloneqq \mathbb{1}} \qquad \frac{}{[\![*]\!] \coloneqq *}$$

$$\frac{}{[\![\bot]\!] \coloneqq \mathbb{0}} \qquad \frac{}{[\![\mathbb{2}]\!] \coloneqq \mathbb{2}}$$

$$\frac{}{[\![\mathbb{1}_2]\!] \coloneqq \mathbb{1}_2} \qquad \frac{}{[\![\mathbb{0}_2]\!] \coloneqq \mathbb{0}_2}$$

$$\frac{}{[\![\text{Sort } u]\!] \coloneqq \text{Sort } u}$$

**Definition 6.9.** Shared values are compiled as such:

$$\frac{}{[\![\text{Shared } A]\!] \coloneqq (!\ [\![A]\!])} \qquad \frac{}{[\![\text{share } Ax]\!] \coloneqq (\&\ [\![x]\!])}$$

**Definition 6.10.** Binders are compiled as such:

$$\frac{[\![A]\!]_{(!\ \mathbb{1})} = A' \quad [\![M]\!]_{(!\ (\Sigma\ \mathbb{1}\ A'))} = M'}{[\![\lambda x : (\text{Shared } A).M]\!] \coloneqq ((!\Lambda\ \mathbb{1}\ A'\ M')(\&\ *))}$$

$$\frac{[\![A]\!] = A' \quad [\![B]\!]_{(!\ (\Sigma\ \mathbb{1}\ A'))} = B'}{[\![\forall x : (\text{Shared } A), B]\!] \coloneqq ((\Pi\ (`\mathbb{1}\ A')\ (`\mathbb{1}\ ((!\Lambda\ \mathbb{1}\ A'\ B')(\&\ *))))*)}$$

$$\frac{[\![A]\!] = A' \quad [\![B]\!]_{(!\ (\Sigma\ \mathbb{1}\ A'))} = B'}{\left[\!\!\left[\sum_{x:\text{Shared } A} B(x)\right]\!\!\right] \coloneqq (\Sigma\ A'\ ((!\Lambda\ \mathbb{1}\ A'\ B')(\&\ \mathbb{1})))}$$

**Definition 6.11.** Application and $\Sigma$-terms are compiled as such:

$$\frac{}{[\![\langle x, y\rangle]\!] \coloneqq ([\![\langle x, y\rangle]\!]_{(!\ \mathbb{1})}(\&\ *))}$$

$$\frac{}{[\![(MN)]\!] \coloneqq ([\![M]\!][\![N]\!])}$$

## 7. Examples

Several examples of type-checking are available in the DCC repository. We have omitted the details here, since the expressions and their inferred types are quite long and not suitable for this medium. We note that DCC is capable of compiling and type-checking the dependent identity function $id : (A : \text{Sort } u) \to (x : A) \to x$:

```
#eval Std.format (dcc_compile (fun (α : Type) (x : α) ⇒ x))
/-
Messages here:
30:0:
((!Λ 𝟙 (`𝟙 (Sort 1))
     (!Λ (Σ 𝟙 (`𝟙 (Sort 1))) (Snd (`𝟙 (Sort 1)))
         (∘ (Snd (Snd (`𝟙 (Sort 1)))) (* (Σ (Σ 𝟙 (`𝟙 (Sort 1))) (Snd (`𝟙 (Sort 1))))))))
 (& *))
-/

#eval (·.fst) <$> Expr.infer (dcc_compile (fun (α : Type) (x : α) ⇒ x))
/-
Messages here:
31:0:
Except.ok (DCC
 ((Π (!<> (`𝟙 (Sort 1)))
     (!Λ 𝟙 (`𝟙 (Sort 1))
         (Π (!<> (Snd (`𝟙 (Sort 1))))
            (!Λ (Σ 𝟙 (`𝟙 (Sort 1))) (Snd (`𝟙 (Sort 1)))
                (∘ (∘ (Snd (`𝟙 (Sort 1))) (Fst (Snd (`𝟙 (Sort 1)))))
                   (* (Σ (Σ 𝟙 (`𝟙 (Sort 1))) (Snd (`𝟙 (Sort 1))))))))))
  (& *)))
-/
```

✔ [Formalized: `Dcc.Compile.Examples`]

Also, some of the usual dependent functions type-check in hand-written DCC. For example, let $f : (x : \mathbb{2}) \to (y : \text{if } x \text{ then } A \text{ else } B) \to y$ where $A : \text{Sort } u$ and $B : \text{Sort } v$. Then, $f$ is expressed in DCC as:

Let $motive \coloneqq (\Lambda\ \mathbb{1}\ (`\mathbb{1}\ \mathbb{2})\ (`(\Sigma\ \mathbb{1}\ \mathbb{2})\ \text{Sort}\ u))$
Let $case_1 \coloneqq (\Lambda\ \mathbb{1}\ (`\mathbb{1}\ \mathbb{2})\ (`(\Sigma\ \mathbb{1}\ \mathbb{2})\ A))$
Let $case_2 \coloneqq (\Lambda\ \mathbb{1}\ (`\mathbb{1}\ \mathbb{2})\ (`(\Sigma\ \mathbb{1}\ \mathbb{2})\ B))$
$f_{A,B} \coloneqq (\Lambda\ \mathbb{2}\ ((\mathbb{2}.Rec\ motive\ case_1\ case_2)*)\ (Snd\ ((\mathbb{2}.Rec\ motive\ case_1\ case_2)*)))$

The application $(f_{\mathbb{1},(=\ (`\mathbb{1}\ \mathbb{1})\ *\ *)}\mathbb{1}_{\mathbb{2}}*)$ type-checks:

```
Messages here:
216:0:
Except.ok (DCC 1)
```

Similarly, the application $(f_{\mathbb{1},(=\ (`\mathbb{1}\ \mathbb{1})\ *\ *)}\mathbb{0}_{\mathbb{2}}((refl\ (`\mathbb{1}\ \mathbb{1}))(\&\ *)(\&\ *)))$ type-checks:

```
Messages here:
224:0:
Except.ok (DCC (= 1 * *))
```

✔ [Formalized: `Dcc.Examples`]

## 8. Comparison to SK Combinators

Previous attempts by others at dependent combinatory type theory [Alt+23b] used the $SK$ combinator calculus. Our approach differs in that it is based on an extension of the simply-typed categorical combinators to dependent types. We believe our success in establishing dependent combinatory type theory is due to the one-to-one correspondence between Curien's categorical combinators [Cur86] and cartesian-closed categories. Our approach also differs from prior ones in that we adapted the categorical combinators to affine logic. As we have demonstrated, this combination leads to the desirable property of computable extensional equality reflection, alongside having decidable type-checking.

## 9. Limitations

We note that the type theory presented in this paper does not have $\eta$-reduction, and due to its affine nature, we have not succeeded in recovering $W$-types. There has been work [FX23] on establishing inductive types in linear logic, and they may be possible through a least-fixedpoint operator. However, they are out of the scope of this paper. Furthermore, we note that due to our eager normalization strategy in judgments, proofs of equalities always have a type of the form $(=\ A\ x\ x)$. However, theorems about equality such as equality being symmetric are still theoretically provable, since type conversion occurs when type-checking applications 4.5.

## 10. Conclusion

Our formalization gives an extension of the simply-typed categorical combinators [Cur86, p. 220] to a dependently-typed, affine setting. To this date, our presentation is the most complete development in dependent combinatory logic: we have a working type-checker written in Lean 4 with decidable type-checking, a procedure for compiling a subset of Lean 4 to that type theory, all objects are well-typed, there are no contexts and no variables, and preservation, progress, consistency, and strong normalization hold, while our type theory is similar in expressivity to the Calculus of Constructions with respect to the $\lambda$-cube. We summarize by emphasizing two properties of this type theory: by extending the categorical combinators to dependent types, we get a simple, but powerful combinatory dependent type theory which feels categorically canonical; and by restricting that type theory to affine logic, we gain computable extensional equality reflection as a definition, rather than a theorem in **Prop**, while having decidable type-checking (a property which is known to be close to impossible to achieve).

## Appendix A. Supplementary Materials

- **Lean 4 formalization repository:**
  https://github.com/lexzaiello/dcc
- **Formalized Reduction rules:**
  https://github.com/lexzaiello/DCC/blob/master/Dcc/Red/Rel.lean#L14
- **Type-checker implementation:**
  https://github.com/lexzaiello/DCC/blob/master/Dcc/Judge.lean
- **Compiler implementation:**
  https://github.com/lexzaiello/DCC/blob/master/Dcc/Compile/Core.lean

## References

[Alt+23a] Thorsten Altenkirch et al. "Combinatory logic and lambda calculus are equal, algebraically". In: *8th International Conference on Formal Structures for Computation and Deduction (FSCD 2023)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2023, pp. 24–1.

[Alt+23b] Thorsten Altenkirch et al. "The Münchhausen Method in Type Theory". In: *28th International Conference on Types for Proofs and Programs (TYPES 2022)*. Ed. by Delia Kesner and Pierre-Marie Pédrot. Vol. 269. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 10:1–10:20. ISBN: 978-3-95977-285-3. DOI: 10.4230/LIPIcs.TYPES.2022.10. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2022.10.

[Bar91] Henk Barendregt. "Introduction to generalized type systems". In: *Journal of Functional Programming* 1.2 (1991), pp. 125–154. DOI: 10.1017/S0956796800020025.

[CCM85] G. Cousineau, P-L. Curien, and M. Mauny. "The categorical abstract machine". In: *Functional Programming Languages and Computer Architecture*. Ed. by Jean-Pierre Jouannaud. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 50–64. ISBN: 978-3-540-39677-2.

[Cur86] P.-L. Curien. "Categorical combinators". In: *Information and Control* 69.1 (1986), pp. 188–254. ISSN: 0019-9958. DOI: https://doi.org/10.1016/S0019-9958(86)80047-X. URL: https://www.sciencedirect.com/science/article/pii/S001999588680047X.

[De 72] Nicolaas Govert De Bruijn. "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem". In: *Indagationes mathematicae (proceedings)*. Vol. 75. 5. Elsevier. 1972, pp. 381–392.

[FX23] Qiancheng Fu and Hongwei Xi. "A calculus of inductive linear constructions". In: *Proceedings of the 8th ACM SIGPLAN International Workshop on Type-Driven Development*. 2023, pp. 1–13.

[Gir87] Jean-Yves Girard. "Linear logic". In: *Theoretical computer science* 50.1 (1987), pp. 1–101.

[Kop01] Alexey P Kopylov. "Decidability of linear affine logic". In: *Information and Computation* 164.1 (2001), pp. 173–198.

[KW84] Jussi Ketonen and Richard Weyhrauch. "A decidable fragment of predicate calculus". In: *Theoretical computer science* 32.3 (1984), pp. 297–307.

[LS88] Joachim Lambek and Philip J Scott. *Introduction to higher-order categorical logic*. Vol. 7. Cambridge University Press, 1988.

[MS84] Per Martin-Löf and Giovanni Sambin. *Intuitionistic type theory*. Vol. 9. Bibliopolis, Naples, 1984.

[New42] Maxwell Herman Alexander Newman. "On theories with a combinatorial definition of" equivalence"". In: *Annals of mathematics* 43.2 (1942), pp. 223–243.

[nLa] nLab authors. *Cartesian closed category*. https://ncatlab.org. Accessed: August 13, 2026. URL: https://ncatlab.org/nlab/show/cartesian+closed+category.

This work is licensed under the Creative Commons Attribution License. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, 171 Second St, Suite 300, San Francisco, CA 94105, USA, or Eisenacher Strasse 2, 10777 Berlin, Germany